HOÀNG QUANG HUY XÂY DỰNG HỆ THỐNG NHẬN DẠNG CỬ CHỈ HÀNH ĐỘNG TAY 2025

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT HƯNG YÊN

HOÀNG QUANG HUY

XÂY DỰNG HỆ THỐNG NHẬN DẠNG CỬ CHỈ HÀNH ĐỘNG TAY

ĐỒ ÁN TỐT NGHIỆP ĐẠI HỌC

HƯNG YÊN - 2025

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT HƯNG YÊN

HOÀNG QUANG HUY

XÂY DỰNG HỆ THỐNG NHẬN DẠNG CỬ CHỈ HÀNH ĐỘNG TAY

NGÀNH: KHOA HỌC MÁY TÍNH

CHUYÊN NGÀNH: KHOA HỌC DỮ LIỆU VÀ TRÍ TUỆ NHÂN TẠO

ĐỒ ÁN TỐT NGHIỆP

NGƯỜI HƯỚNG DẪN	**CHỦ TỊCH HỘI ĐỒNG**
(Ký và ghi rõ họ tên)	*(Ký và ghi rõ họ tên)*

HƯNG YÊN – 2025

NHẬN XÉT

Nhận xét của giáo viên hướng dẫn

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

GIÁO VIÊN HƯỚNG DẪN

LỜI CAM ĐOAN

Em xin cam đoan Đồ án tốt nghiệp "Xây dựng hệ thống nhận dạng cử chỉ hành động tay" là sản phẩm của em. Những phần sử dụng tài liệu tham khảo trong đồ án đã được nêu rõ trong phần tài liệu tham khảo. Các số liệu, kết quả trình bày trong đồ án là hoàn toàn trung thực, nếu sai em xin chịu hoàn toàn trách nhiệm và chịu mọi kỷ luật của bộ môn và nhà trường đề ra.

Hưng yên, ngày 07 tháng 06 năm 2025

Sinh viên

Huy

Hoàng Quang Huy

LỜI CẢM ƠN

Trước tiên, em xin gửi lời biết ơn sâu sắc đến Bộ môn Khoa học Máy tính, Khoa Công nghệ Thông tin – Trường Đại học Sư phạm Kỹ thuật Hưng Yên đã tạo điều kiện rất tốt cho em trong suốt quá trình thực hiện đồ án tốt nghiệp này. Nhờ sự hỗ trợ tận tình từ Bộ môn, em đã có cơ hội tiếp cận với nhiều kiến thức mới, cũng như thực hành các kỹ năng chuyên môn một cách bài bản và hiệu quả. Môi trường học tập tại Bộ môn chính là nền tảng vững chắc giúp em hoàn thành tốt nhiệm vụ nghiên cứu của mình.

Em cũng xin gửi lời cảm ơn chân thành nhất tới Thầy Lê Trung Hiếu, người đã dành nhiều thời gian quý báu, kiến thức và kinh nghiệm để hướng dẫn em từng bước trong suốt quá trình làm đồ án. Những lời khuyên, sự chỉ bảo tận tâm và động viên của Thầy đã giúp em vượt qua những khó khăn, thử thách, đồng thời tạo động lực để em cố gắng hoàn thiện bản thân và sản phẩm nghiên cứu. Thầy chính là người thầy, người hướng dẫn tận tụy mà em rất trân trọng.

Bên cạnh đó, em xin gửi lời cảm ơn sâu sắc tới toàn thể các Thầy Cô trong trường đã luôn quan tâm, dạy dỗ và truyền đạt cho em những kiến thức quý báu suốt những năm học vừa qua. Những bài giảng bổ ích, những lời khuyên chân thành và sự tận tình giúp đỡ của các Thầy Cô đã góp phần không nhỏ vào việc hình thành nền tảng tri thức và kỹ năng cho em ngày hôm nay.

Mặc dù em đã cố gắng rất nhiều trong quá trình hoàn thành đồ án, nhưng do trình độ, kinh nghiệm còn hạn chế, khó tránh khỏi những thiếu sót và sai sót nhất định. Em rất mong nhận được sự góp ý, phản hồi từ các Thầy Cô để có thể khắc phục, sửa chữa và nâng cao hơn nữa trong tương lai. Em cam kết sẽ tiếp thu nghiêm túc mọi ý kiến, đánh giá với tinh thần cầu tiến và học hỏi không ngừng để hoàn thiện bản thân và công trình nghiên cứu của mình.

Một lần nữa, em xin chân thành cảm ơn và tri ân tất cả những sự giúp đỡ quý báu từ các Thầy Cô, Bộ môn và Nhà trường đã đồng hành cùng em trong hành trình học tập và nghiên cứu này.

MỤC LỤC

MỤC LỤC 6
DANH SÁCH CÁC THUẬT NGỮ 8
DANH SÁCH BẢNG BIỂU 9
DANH SÁCH HÌNH VẼ 10
CHƯƠNG 1:TỔNG QUAN VỀ ĐỀ TÀI 10
1.1 Lý do chọn đề tài 11
1.2 Mục tiêu của đề tài 11
1.3 Giới hạn và phạm vi của đề tài 12
1.4 Kết quả dự kiến đạt được 13
CHƯƠNG 2: TỔNG QUAN VỀ CÁC MÔ HÌNH THUẬT TOÁN TRONG NHẬN DẠNG CỬ CHỈ HÀNH ĐỘNG TAY 14
2.1. Nhận diện cử chỉ hành động tay 14
2.2. Mô hình học sâu (Deep Learning Models) 15
2.2.1 3-Dimensional Convolutional Neural Networks (3D CNN) 15
2.2.2 Recurrent Neural Network(RNN): 17
2.2.3. Long Short-Term Memory (LSTM): 19
2.2.4 Gated Recurrent Unit (GRU): 21
2.2.5 Kiến trúc Hybrid 3D-CNN và RNN/LSTM/GRU 22
2.2.6 Ma trận đánh giá: 25
CHƯƠNG 3: XÂY DỰNG VÀ HUẤN LUYỆN MÔ HÌNH TRONG HỆ THỐNG NHẬN DẠNG CỬ CHỈ HÀNH ĐỘNG TAY 27
3.1 Khảo sát và thu thập dữ liệu 27
3.1.1 Khảo sát hiện trạng và xác định yêu cầu hệ thống 27

3.1.1 Thu thập và xử lý dữ liệu 27
3.2 Kết quả thực nghiệm và phân tích hiệu suất mô hình 30
3.2.1 Kết quả huấn luyện của 3DCNN + RNN trong 3 trường hợp.....31
3.2.2 Kết quả huấn luyện của 3DCNN+LSTM trong 3 trường hợp: 35
3.2.3 Kết quả huấn luyện của 3DCNN+GRU trong 3 trường hợp:.....37
3.2.4 Tổng hợp kết quả mô hình và phân tích hiệu suất tốt nhất 40
CHƯƠNG 4: TRIỂN KHAI XÂY DỰNG ỨNG DỤNG NHẬN DẠNG CỬ CHỈ HÀNH ĐỘNG TAY 43
4.1 Mục đích.....43
4.2 Công nghệ sử dụng.....43
4.3 Cáu trúc và tính năng chính của ứng dụng.....45
4.4 Quy trình xử lý và dự đoán.....47
4.5 Các điểm nổi bật trong thiết kế.....48
4.6 Kết luận.....48
KẾT LUẬN 49
TÀI LIỆU THAM KHẢO.....51

DANH SÁCH CÁC THUẬT NGỮ

Từ viết tắt	Từ đầy đủ	Giải thích
3DCNN	3-Dimensional Convolutional Neural Network	Nó là một loại mạng tích chập (CNN) mở rộng thêm chiều thời gian hoặc chiều thứ 3, thường dùng để xử lý dữ liệu không gian-thời gian như video, chuỗi ảnh hoặc dữ liệu 3D.
RNN	Recurrent Neural Network	Mạng nơ-ron hồi tiếp, xử lý dữ liệu chuỗi bằng cách ghi nhớ thông tin qua các bước thời gian.
LSTM	Long Short-Term Memory	Mạng nhớ dài hạn ngắn hạn, một biến thể của RNN giúp lưu giữ thông tin dài hạn hiệu quả hơn.
GRU	Gated Recurrent Unit	Đơn vị hồi tiếp có cửa điều khiển, biến thể của RNN tương tự LSTM nhưng đơn giản hơn và nhanh hơn.

DANH SÁCH BẢNG BIỂU

Bảng 3. 1:Số lượng của các cử chỉ tay 28
Bảng 3. 2:Số lượng tập huấn luyện,kiểm thử và dự đoán 28
Bảng 3. 3: Các bước tiền xử lý trong VideoDataGenerator 29
Bảng 3. 4: Các bộ tham số thử nghiệm 31
Bảng 3. 5: Kết quả huấn luyện của 3DCNN + RNN trong 3 trường hợp 32
Bảng 3. 6: Kết quả huấn luyện của 3DCNN+LSTM trong 3 trường hợp 35
Bảng 3. 7: Kết quả huấn luyện của 3DCNN+GRU trong 3 trường hợp 38
Bảng 3. 8: Đánh giá kết quả tốt nhất của các mô hình 3DCNN + RNN, LSTM và GRU 41
Bảng 3. 9: Bảng đánh giá hiệu suất nhận dạng các lớp cử chỉ hành động tay của mô hình 3DCNN + LSTM 42

DANH SÁCH HÌNH VẼ

Hình 2. 1:Cử chỉ tĩnh và cử chỉ động [11].....15
Hình 2. 2:Kiến trúc của mô hình 3DCNN [9].....16
Hình 2. 3:Kiến trúc của mạng RNN [4].....17
Hình 2. 4:Kiến trúc mô hình LSTM [5].....19
Hình 2. 5:Kiến trúc mô hình GRU[7].....21
Hình 2. 6:Hybrid 3DCNN+RNN/LSTM/GRU.....23
Hình 3. 1: Biểu đồ Accuracy và Loss trong quá trình huấn luyện và xác thực của mô hình 3DCNN + RNN — Trường hợp 1.....33
Hình 3. 2: Biểu đồ Accuracy và Loss trong quá trình huấn luyện và xác thực của mô hình 3DCNN + RNN — Trường hợp 2.....33
Hình 3. 3: Biểu đồ Accuracy và Loss trong quá trình huấn luyện và xác thực của mô hình 3DCNN + RNN — Trường hợp 3.....34
Hình 3. 4: Biểu đồ Accuracy và Loss trong quá trình huấn luyện và xác thực của mô hình 3DCNN + LSTM — Trường hợp 1.....36
Hình 3. 5: Biểu đồ Accuracy và Loss trong quá trình huấn luyện và xác thực của mô hình 3DCNN + LSTM — Trường hợp 2.....36
Hình 3. 6: Biểu đồ Accuracy và Loss trong quá trình huấn luyện và xác thực của mô hình 3DCNN + LSTM — Trường hợp 3.....37
Hình 3. 7: Biểu đồ Accuracy và Loss trong quá trình huấn luyện và xác thực của mô hình 3DCNN + GRU — Trường hợp 1.....39
Hình 3. 8: Biểu đồ Accuracy và Loss trong quá trình huấn luyện và xác thực của mô hình 3DCNN + GRU — Trường hợp 2.....39
Hình 3. 9: Biểu đồ Accuracy và Loss trong quá trình huấn luyện và xác thực của mô hình 3DCNN + GRU — Trường hợp 3.....40
Hình 4. 1: Demo của ứng dụng.....46

CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI

1.1 Lý do chọn đề tài

Công nghệ điều khiển bằng cử chỉ hành động tay đã thúc đẩy mạnh mẽ việc phát triển các phương thức tương tác người-máy không tiếp xúc, mang lại sự an toàn và tiện lợi cho người dùng. Các ứng dụng nhận diện cử chỉ động hiện được sử dụng rộng rãi trong nhiều lĩnh vực như y tế, dịch vụ ăn uống, giải trí, điện thoại thông minh và ngành công nghiệp ô tô. Trong dự án này, mô hình được huấn luyện chuyên biệt để nhận dạng các cử chỉ hành động tay. Mô hình có khả năng dự đoán các động tác tay theo thời gian thực thông qua webcam, từ đó gửi các phím tắt tương ứng để điều khiển các tác vụ trên máy tính. Giải pháp này giúp người dùng thực hiện các thao tác mà không cần chạm vào thiết bị, nâng cao hiệu quả công việc và cải thiện trải nghiệm tương tác.

1.2 Mục tiêu của đề tài

Mục tiêu tổng quát:

Nghiên cứu và phát triển mô hình AI nhằm xây dựng hệ thống nhận diện chính xác các cử chỉ hành động tay động từ video, sử dụng công nghệ Deep Learning. Mô hình được thiết kế để xử lý dữ liệu video thời gian thực, tận dụng mạng nơ-ron sâu kết hợp 3DCNN và RNN/LSTM/GRU để trích xuất đặc trưng không gian-thời gian hiệu quả.

Ứng dụng demo được phát triển dưới dạng phần mềm desktop với giao diện đồ họa thân thiện sử dụng PyQt6, cho phép người dùng điều khiển và nhận diện cử chỉ tay trực tiếp qua webcam. Kết quả dự đoán được hiển thị tức thì trên giao diện, hỗ trợ tương tác người-máy không tiếp xúc.

Quá trình kiểm thử và tối ưu hóa mô hình được thực hiện trên tập dữ liệu thực tế với nhiều lớp cử chỉ khác nhau, nhằm đảm bảo độ chính xác và sự ổn định của hệ thống trong môi trường thực tế.

Mục tiêu cụ thể:

Phát triển mô hình học sâu (Deep Learning) kết hợp 3D-CNN và mạng tuần tự (RNN, LSTM, GRU) để nhận diện chính xác các cử chỉ tay động phổ biến từ dữ liệu

video và hình ảnh. Mô hình được thiết kế nhằm khai thác hiệu quả đặc trưng không gian – thời gian của chuỗi khung hình video thời gian thực, phục vụ việc nhận diện các cử chỉ tay với độ chính xác cao.

Thực hiện kiểm thử và đánh giá mô hình trên bộ dữ liệu 20BN-Jester gồm 6 lớp cử chỉ tay tiêu biểu với hơn 12.000 video, đồng thời áp dụng các kỹ thuật tối ưu thuật toán và điều chỉnh tham số nhằm đảm bảo mô hình hoạt động ổn định, chính xác trên các tập dữ liệu thực tế khác nhau.

Xây dựng và tích hợp mô hình nhận diện vào ứng dụng phần mềm desktop sử dụng PyQt6 và OpenCV, cho phép xử lý luồng video trực tiếp từ webcam, nhận dạng cử chỉ tay ngay lập tức. Hệ thống hỗ trợ tương tác không tiếp xúc, nâng cao trải nghiệm người dùng trong các ứng dụng thực tế với điều kiện ánh sáng và góc nhìn đa dạng.

1.3 Giới hạn và phạm vi của đề tài

Đề tài này tập trung vào việc nhận diện một số lượng hạn chế các cử chỉ hành động tay phổ biến, cụ thể là 6 lớp cử chỉ tiêu biểu từ bộ dữ liệu 20BN-Jester. Do đó, phạm vi nghiên cứu chưa bao phủ toàn bộ các loại cử chỉ đa dạng có thể xuất hiện trong thực tế. Điều này giúp tập trung phát triển mô hình cho những cử chỉ phổ biến nhưng cũng đặt ra giới hạn về khả năng áp dụng rộng rãi cho các động tác phức tạp hoặc ít gặp hơn.

Dữ liệu huấn luyện được sử dụng trong đề tài là từ các tập dữ liệu có sẵn và có hạn. Mặc dù bộ dữ liệu đã cung cấp nhiều video với các tình huống đa dạng, nhưng vẫn có thể chưa phản ánh đầy đủ những biến thể về điều kiện ánh sáng, góc nhìn và môi trường thực tế. Việc này dẫn đến mô hình có thể chưa hoàn toàn phù hợp hoặc thiếu chính xác khi áp dụng trong các điều kiện khác biệt hoặc phức tạp hơn so với dữ liệu huấn luyện.

Tài nguyên tính toán, đặc biệt là GPU, cũng là một hạn chế lớn. Mô hình học sâu cần sức mạnh xử lý cao nhưng phần cứng hiện tại chưa đáp ứng đủ nên việc tối ưu và mở rộng mô hình gặp khó khăn. Vì vậy, em phải chọn kiến trúc mô hình phù hợp với

khả năng máy, điều này hạn chế việc áp dụng các mô hình phức tạp hoặc dữ liệu lớn sau này.

Do thời gian nghiên cứu và phần cứng hạn chế, em chỉ phát triển hệ thống trên nền tảng desktop và chưa mở rộng sang thiết bị hay môi trường khác. Điều này làm giảm khả năng ứng dụng hệ thống trong thực tế.

1.4 Kết quả dự kiến đạt được

Đề tài hướng tới phát triển một mô hình học sâu có khả năng nhận diện chính xác các cử chỉ tay phổ biến, tập trung vào 6 lớp cử chỉ tiêu biểu được lựa chọn. Mô hình dự kiến đạt được hiệu suất cao trong việc nhận diện các cử chỉ tay này từ dữ liệu video đã được chia thành các frames ảnh, đáp ứng yêu cầu về độ chính xác và khả năng nhận dạng trong môi trường thực tế đa dạng.

Bên cạnh đó, mô hình sẽ được tích hợp vào một ứng dụng desktop, sử dụng webcam để thu nhận hình ảnh và video từ người dùng. Ứng dụng này sẽ cho phép nhận diện cử chỉ tay theo thời gian thực, tức thì phản hồi kết quả dự đoán ngay khi người dùng thực hiện động tác. Điều này sẽ góp phần nâng cao trải nghiệm tương tác không tiếp xúc và mở rộng khả năng ứng dụng trong nhiều lĩnh vực thực tế như giao tiếp, điều khiển thiết bị thông minh hoặc giáo dục.

Ngoài ra, hệ thống sẽ được kiểm thử và đánh giá kỹ lưỡng trên các tập dữ liệu thực tế để đảm bảo tính ổn định và độ chính xác cao trong nhiều điều kiện khác nhau như thay đổi ánh sáng, góc nhìn hay chất lượng hình ảnh. Dựa trên kết quả đánh giá, nhóm nghiên cứu sẽ đề xuất các hướng cải tiến, tối ưu thuật toán và mô hình nhằm nâng cao hiệu suất nhận diện, đồng thời giảm thiểu các sai số, đảm bảo ứng dụng có thể hoạt động hiệu quả và bền vững trong thực tế.

CHƯƠNG 2: TỔNG QUAN VỀ CÁC MÔ HÌNH THUẬT TOÁN TRONG NHẬN DẠNG CỬ CHỈ HÀNH ĐỘNG TAY

2.1 Nhận diện cử chỉ hành động tay

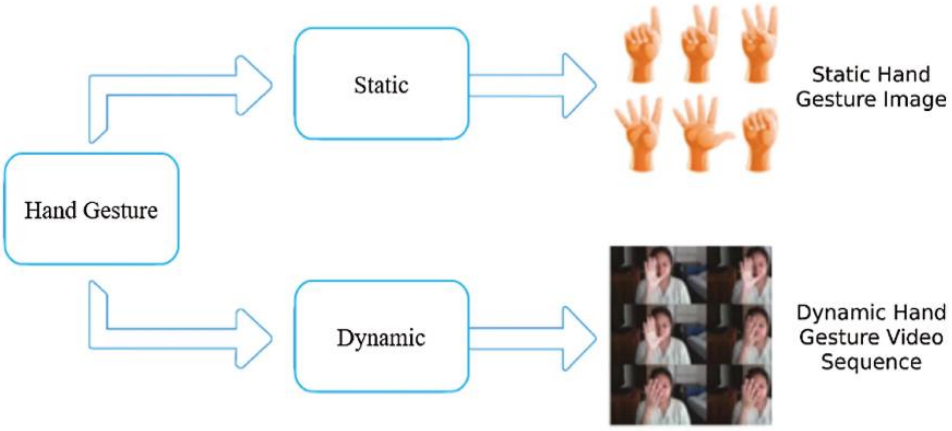
Nhận diện cử chỉ hành động tay là một lĩnh vực chuyên sâu trong thị giác máy tính và nhận diện hình ảnh, tập trung vào việc phát hiện, nhận diện và hiểu các chuyển động của bàn tay trong không gian và thời gian. Trong lĩnh vực này, hệ thống máy tính được thiết kế để phân tích các chuỗi hình ảnh hoặc video nhằm nhận biết các động tác tay mà con người thực hiện, từ đó có thể áp dụng trong nhiều lĩnh vực như giao tiếp người-máy, điều khiển thiết bị không tiếp xúc, hoặc các ứng dụng tương tác thông minh.

Cử chỉ tay được chia thành hai loại chính, bao gồm cử chỉ tĩnh và cử chỉ động. Cử chỉ tĩnh là những tư thế hay dáng tay không thay đổi theo thời gian, ví dụ như nắm tay, hoặc tạo các ký hiệu số bằng tay. Các mô hình nhận diện cử chỉ tĩnh thường chỉ cần phân tích một hoặc vài khung hình đơn lẻ để xác định được tư thế tay đó, do không cần quan tâm đến biến động hay chuyển động trong chuỗi video.

Ngược lại, cử chỉ động là những hành động tay có sự thay đổi liên tục theo thời gian, ví dụ như vuốt lên, bắt tay, thu nhỏ màn hình với 2 ngón tay hay các chuỗi động tác phức tạp hơn. Để nhận diện chính xác các cử chỉ động, hệ thống cần phân tích toàn bộ chuỗi khung hình trong video, từ đó nắm bắt được sự biến đổi liên tục và các đặc trưng thời gian của chuyển động. Việc này đòi hỏi các mô hình nhận diện không chỉ xử lý thông tin không gian trên từng khung hình mà còn phải học được mối quan hệ tuần tự giữa các khung hình theo thời gian, giúp phân biệt chính xác các hành động dù có sự biến đổi về tốc độ, hướng di chuyển hay điều kiện môi trường.

Trong đề tài này, trọng tâm được đặt vào nhận diện cử chỉ hành động tay thuộc nhóm cử chỉ động, với mục tiêu xây dựng các mô hình có khả năng nhận diện chính xác các hành động tay phổ biến dựa trên dữ liệu video. Việc xử lý chuỗi hình ảnh động này yêu cầu áp dụng các kỹ thuật học sâu kết hợp giữa mạng tích chập 3 chiều (3D-CNN) để trích xuất đặc trưng không gian-thời gian và các mạng tuần tự như RNN, LSTM hay

GRU để học các đặc trưng tuần tự phức tạp. Nhờ đó, hệ thống có thể nhận diện các cử chỉ tay một cách hiệu quả, đáp ứng yêu cầu về độ chính xác và khả năng ứng dụng trong thực tế [11].

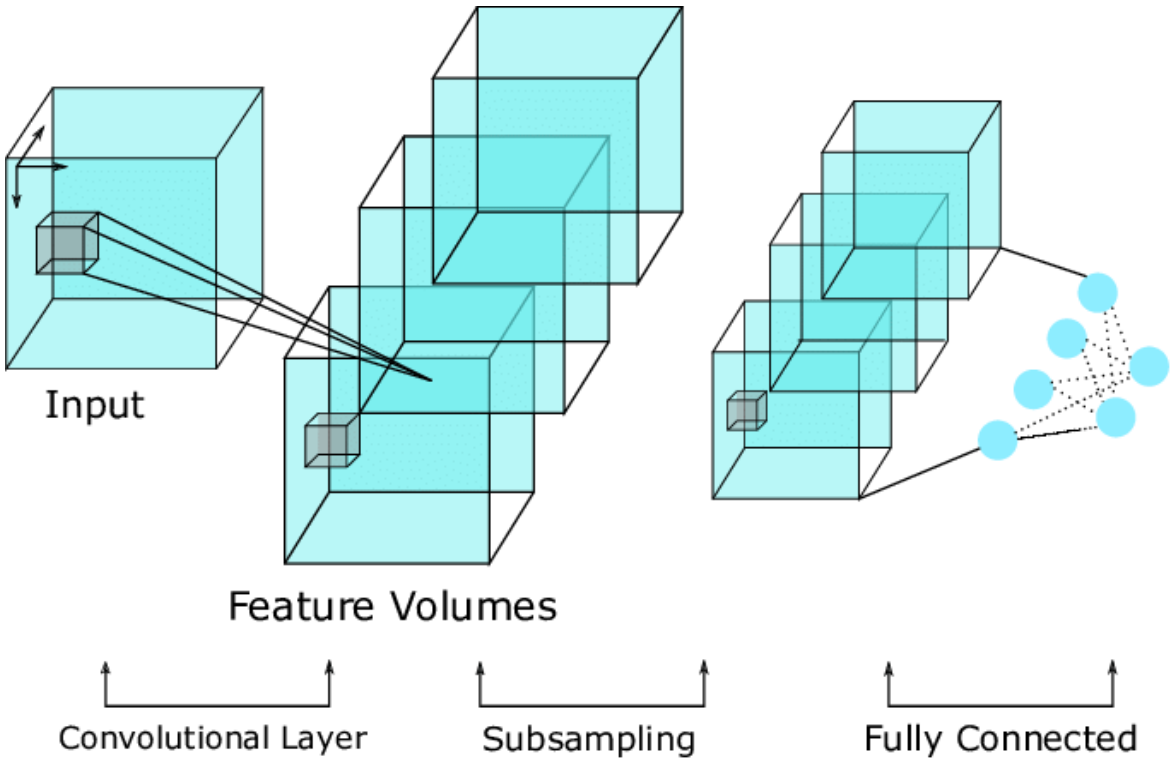


Hình 2. 1:Cử chỉ tĩnh và cử chỉ động [11]

2.2. Mô hình học sâu (Deep Learning Models)

2.2.1 3-Dimensional Convolutional Neural Networks (3D CNN)

Khác với CNN truyền thống chỉ xử lý hình ảnh 2D, 3D CNN có khả năng xử lý dữ liệu video, bao gồm cả không gian và thời gian. Điều này giúp mô hình nhận diện được các cử chỉ tay động trong video, vì dữ liệu video chứa thông tin về chuyển động và các đặc trưng thay đổi theo thời gian. 3D CNN mở rộng mạng CNN để nhận diện không chỉ các đặc trưng không gian mà còn cả các đặc trưng thời gian của cử chỉ tay trong video. Cấu trúc này giúp mô hình nhận diện chính xác hơn các cử chỉ tay động như vẫy tay, bắt tay mang lại hiệu suất tốt hơn so với CNN 2D khi xử lý video. Do đó, 3D CNN là một công cụ mạnh mẽ trong việc nhận diện cử chỉ tay động trong môi trường video [1,2].



Hình 2. 2:Kiến trúc của mô hình 3DCNN [9]

Cấu trúc của 3D CNN:

Input: Dữ liệu đầu vào có dạng 3D, ví dụ: video (kích thước: chiều cao x chiều rộng x số khung hình).

Conv3D layers: Các lớp convolution (chập) được mở rộng để xử lý dữ liệu 3D bằng cách sử dụng bộ lọc 3D (tương tự bộ lọc 2D, nhưng mở rộng thêm chiều thứ ba).

Max Pooling 3D: Tiến hành giảm kích thước của dữ liệu bằng cách chọn giá trị cực đại trong các khối 3D.

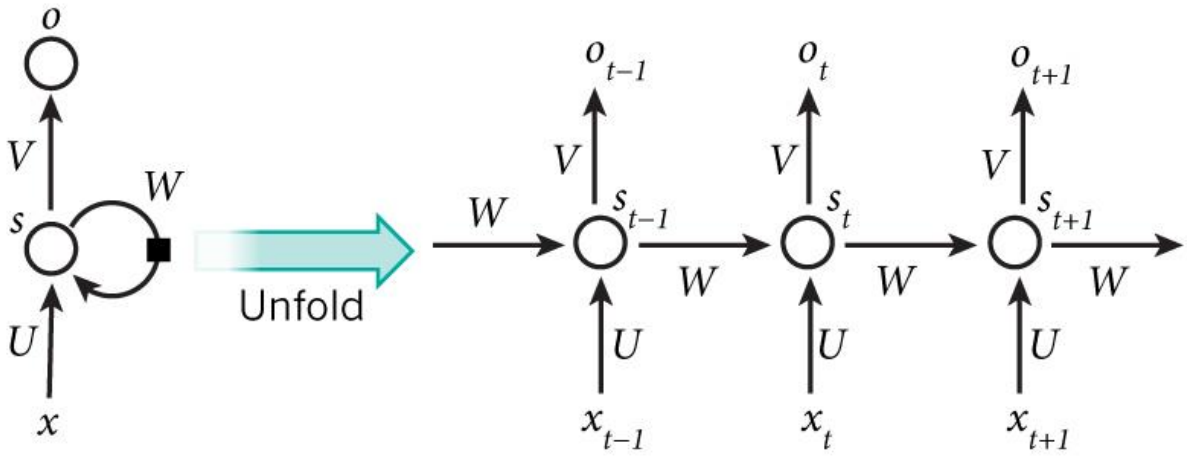
Fully Connected layers: Sau khi các đặc trưng được trích xuất từ các lớp convolution và pooling, dữ liệu được kết nối đầy đủ để phân loại hoặc dự đoán.

Ưu điểm: Mô hình này có thể nắm bắt thông tin không gian và thời gian từ dữ liệu 3D, đặc biệt hữu ích trong các bài toán như nhận dạng cử chỉ tay, phân tích video, hoặc dự đoán các bệnh lý từ dữ liệu y tế.

2.2.2 Recurrent Neural Network(RNN):

Mạng Nơ-ron Hồi tiếp (Recurrent Neural Network - RNN) là một loại mạng nơ-ron được thiết kế đặc biệt để xử lý dữ liệu dạng chuỗi tuần tự. Khác với các mạng nơ-ron thông thường chỉ làm việc với dữ liệu tĩnh, RNN có khả năng lưu giữ thông tin từ những bước thời gian trước nhờ vào trạng thái ẩn (hidden state). Điều này giúp mô hình có thể ghi nhớ và học các mối liên hệ theo trình tự trong chuỗi dữ liệu, rất phù hợp với các bài toán như xử lý ngôn ngữ, phân tích video hay các chuỗi tín hiệu thời gian. [4]

Cấu trúc RNN:



Hình 2. 3:Kiến trúc của mạng RNN [4]

Về cấu trúc, RNN được xem là một mạng có vòng lặp, ở đó trạng thái ẩn của bước trước sẽ được truyền sang bước tiếp theo. Khi biểu diễn theo dạng mở rộng thời gian (unfolded), mô hình sẽ là một chuỗi các mạng nơ-ron liên tiếp tương ứng với từng bước *t* trong chuỗi dữ liệu. Mỗi bước sẽ nhận một đầu vào riêng và trạng thái ẩn từ bước

trước, sau đó tính toán trạng thái ẩn mới. Nhờ vậy, RNN có thể nhớ lại thông tin trong quá khứ và kết hợp với dữ liệu hiện tại để đưa ra dự đoán.

Cụ thể hơn, tại bước thời gian *t,* mô hình sẽ nhận đầu vào x_t cùng với trạng thái ẩn h_{t-1} của bước trước đó. Từ đó, nó sẽ tính ra trạng thái ẩn mới h_t, dùng để dự đoán và cũng được truyền cho bước kế tiếp. Quá trình này giúp mạng học được các đặc trưng tuần tự, nhưng do vấn đề vanishing gradient, RNN truyền thống gặp khó khăn trong việc ghi nhớ các thông tin ở xa trong chuỗi.

Quá trình này giúp mạng ghi nhớ thông tin trong chuỗi, từ đó học được các phụ thuộc dài hạn (nhưng với giới hạn nhất định do vanishing gradient).

Tại mỗi bước thời gian *t,* mô hình nhận đầu vào x_t và trạng thái ẩn h_{t-1} từ bước trước sau đó tính trạng thái ẩn mới $h_t$theo công thức:

$$h_t = f(W_{xh}x_t + W_{hh}h_{t-1} + b_h) \quad (2.1)$$

Trong đó:

h_t trạng thái ẩn tại thời điểm t.

x_t đầu vào tại thời điểm *t.*

W_{xh} là ma trận trọng số nhân với trạng thái ẩn.

W_{hh} ma trận trọng số kết nối trạng thái ẩn trước đó với trạng thái ẩn hiện tại.

$b_h$tham số bias cho trạng thái ẩn.

f hàm kích hoạt (thường là hàm tanh hoặc ReLU).

Đầu ra tại bước *t*

Trạng thái ẩn h_t được dùng để tính đầu ra y_t (hình tròn màu xanh dương) bằng:

$$y_t = W_{hy}h_t + b_t \quad (2.2)$$

Trong đó:

y_t: Đầu ra tại thời điểm t.

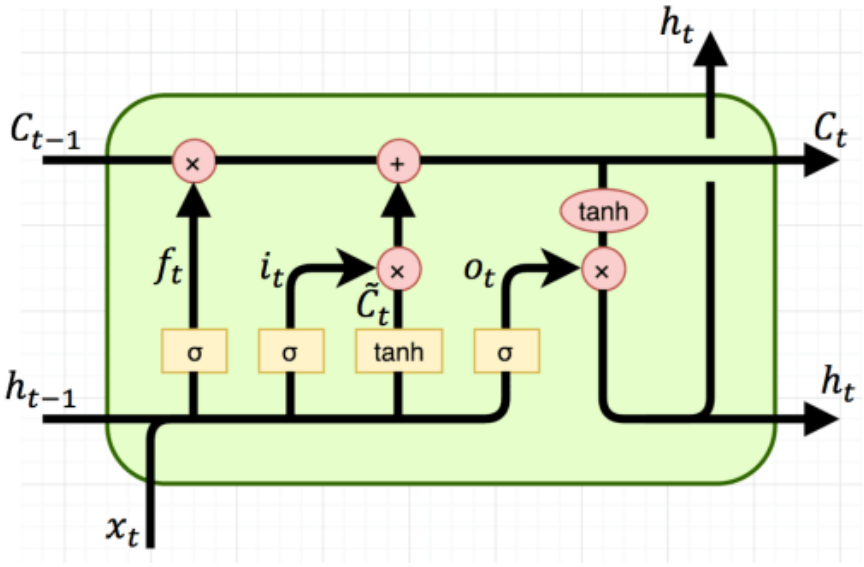
W_{hy}: Ma trận trọng số kết nối trạng thái ẩn với đầu ra.

b_t: Tham số bias cho đầu ra

Vì vậy, mặc dù RNN có thể xử lý dữ liệu tuần tự rất hiệu quả, nhưng vẫn tồn tại hạn chế. Những mạng nơ-ron biến thể như LSTM hay GRU được phát triển để khắc phục những nhược điểm này, giúp cải thiện khả năng ghi nhớ dài hạn và hiệu quả học tập của mô hình.

2.2.3. Long Short-Term Memory (LSTM):

Mô hình Long Short-Term Memory (LSTM) là một biến thể đặc biệt của mạng nơ-ron hồi tiếp (RNN), được thiết kế để khắc phục vấn đề vanishing gradient mà các RNN truyền thống thường gặp phải khi xử lý các chuỗi dữ liệu dài. Nhờ cấu trúc đặc biệt, LSTM có khả năng ghi nhớ cả thông tin dài hạn lẫn ngắn hạn trong chuỗi dữ liệu, giúp mô hình học và dự đoán chính xác hơn trong những bài toán liên quan đến chuỗi thời gian, ví dụ như phân tích văn bản, nhận dạng giọng nói hay dự báo thời tiết [5, 6].



Hình 2. 4:Kiến trúc mô hình LSTM [5]

Về cấu trúc, LSTM được xây dựng dựa trên các cổng (gates) dùng để kiểm soát luồng thông tin đi qua mạng. Cụ thể:

Forget gate (cổng quên): Giúp mô hình quyết định phần thông tin nào trong trạng thái trước cần được loại bỏ, tránh lưu giữ những dữ liệu không còn cần thiết.

Input gate (cổng nhập): Xác định những thông tin mới nào sẽ được cập nhật và ghi vào trạng thái tế bào (cell state), giúp mô hình bổ sung kiến thức mới vào bộ nhớ.

Output gate (cổng xuất): Điều khiển thông tin nào từ trạng thái tế bào sẽ được xuất ra làm đầu ra của mạng tại bước thời gian hiện tại.

Các cổng này được tính toán thông qua các công thức:

$$f_t=\sigma(U_f*x_t+W_f*h_{t-1}+b_f) \quad (2.3)$$

$$i_t=\sigma(U_i*x_t+W_i*h_{t-1}+b_i) \quad (2.4)$$

$$o_t = \sigma(U_o * x_t + W_o * h_{t-1} + b_o) \quad (2.5)$$

Trong đó:

x_t là đầu vào tại bước thời gian t.

h_{t-1} là trạng thái ẩn tại bước thời gian trước đó.

U_f, U_i, U_o là các ma trận trọng số nhân với đầu vào x_t.

W_f, W_i, W_o là các ma trận trọng số nhân với trạng thái ẩn h_{t-1}.

b_f, b_i, b_o là các vector bias.

Hàm sigmoid σ giúp giá trị các cổng nằm trong khoảng (0,1), xác định mức độ mở hay đóng của cổng.

Tiếp theo, trạng thái tế bào ứng viên được tính bằng:

$$\widetilde{c_t} = \tanh(U_c * x_t + W_c * h_{t-1} + b_c) \quad (2.6)$$

Trạng thái tế bào được cập nhật theo công thức:

$$c_t=f_t*c_{t-1}+i_t*\widetilde{c_t} \quad (2.7)$$

Và trạng thái ẩn đầu ra được tính như sau:

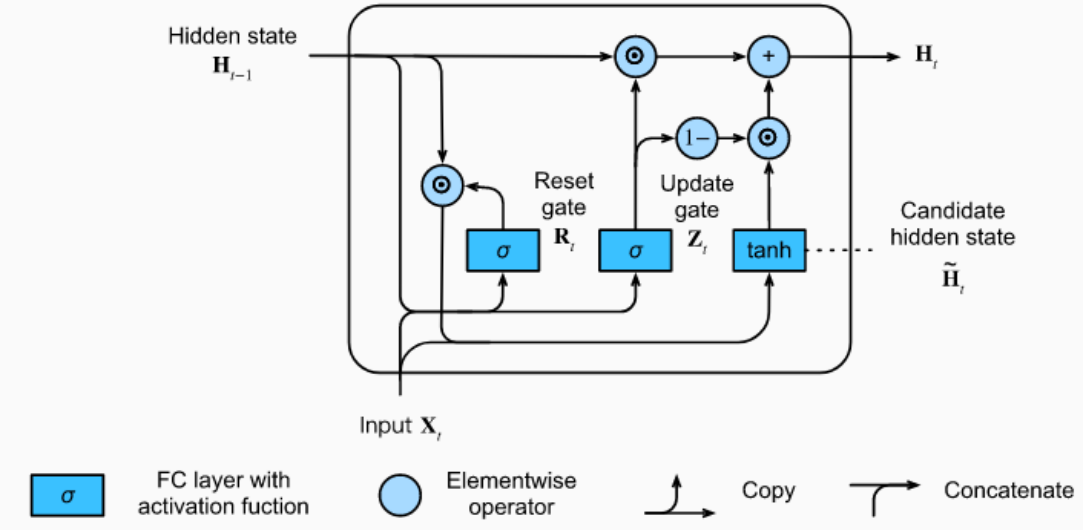
$$h_t = o_t * tanh(c_t) \quad (2.8)$$

Các cổng này cho phép LSTM duy trì thông tin qua nhiều bước thời gian và điều chỉnh độ quan trọng của thông tin tại mỗi bước.

Ưu điểm: LSTM rất hiệu quả trong việc xử lý và dự đoán các chuỗi dài, và có thể học được các mối quan hệ phụ thuộc dài hạn trong dữ liệu. Nó cũng giải quyết vấn đề mất thông tin trong các mô hình RNN truyền thống.

2.2.4 Gated Recurrent Unit (GRU):

Mô hình **Gated Recurrent Unit (GRU)** là một biến thể của mạng nơ-ron hồi tiếp (RNN), được phát triển để giải quyết vấn đề vanishing gradient và cải thiện khả năng học các chuỗi dài. GRU có kiến trúc tương tự như LSTM nhưng đơn giản hơn, giảm bớt số lượng cổng, làm cho mô hình GRU dễ dàng và nhanh chóng hơn trong việc huấn luyện [7].



Hình 2. 5:Kiến trúc mô hình GRU[7]

Cấu trúc GRU:

GRU bao gồm hai cổng chính:

Reset Gate (cổng đặt lại): Quyết định mức độ của trạng thái cũ sẽ bị quên khi tính toán trạng thái mới.

$$r_t = \sigma(U_r x_t + W_r h_{t-1} + b_r) \quad (2.9)$$

Update Gate (cổng cập nhật): Quyết định mức độ mà trạng thái cũ sẽ được giữ lại và mức độ mà trạng thái mới sẽ được tính toán.

$$z_t = \sigma(U_z x_t + W_z h_{t-1} + b_z) \quad (2.10)$$

Trong đó:

x_t là đầu vào tại bước thời gian t.

h_{t-1} là trạng thái ẩn tại bước thời gian trước đó.

U_z, U_r là các ma trận trọng số nhân với đầu vào x_t.

W_z, W_r là các ma trận trọng số nhân với trạng thái ẩn h_{t-1}.

b_z, b_r là các vector bias.

Trạng thái ứng viên được tính bằng:

$$\widetilde{h_t} = \tanh(U_h xt + W_h(r_t * h_{t-1}) + b_h) \quad (2.11)$$

Cập nhật trạng thái ẩn:

$$h_t = (1 - z_t) * h_{t-1} + z_t * \widetilde{h_t} \quad (2.12)$$

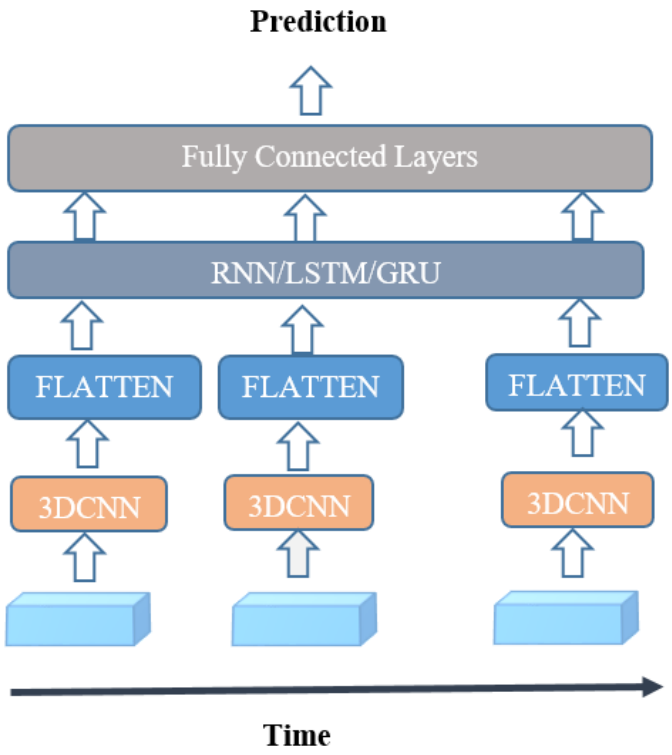
Các cổng này giúp mô hình GRU xác định thông tin quan trọng cần được lưu trữ và thông tin nào cần được quên, tương tự như LSTM nhưng với cấu trúc đơn giản hơn, chỉ sử dụng hai cổng thay vì ba.

Ưu điểm: GRU thường có hiệu suất huấn luyện nhanh hơn và yêu cầu ít tài nguyên tính toán hơn so với LSTM, đồng thời vẫn giữ được khả năng học tốt từ các chuỗi dài.

2.2.5 Kiến trúc Hybrid 3D-CNN và RNN/LSTM/GRU

Để xử lý các dữ liệu dạng chuỗi không gian-thời gian như video hoặc chuỗi ảnh động, một phương pháp phổ biến và hiệu quả là kết hợp mạng tích chập 3 chiều (3D-CNN) với các mô hình mạng hồi tiếp như RNN, LSTM hoặc GRU. Kiến trúc này tận

dụng ưu điểm mạnh mẽ của cả hai loại mạng, nhằm trích xuất và học các đặc trưng phức tạp đồng thời theo không gian và thời gian [10, 11, 12].



Hình 2. 6:Hybrid 3DCNN+RNN/LSTM/GRU

3D-CNN:

3D-CNN được thiết kế đặc biệt để trích xuất các đặc trưng không gian và thời gian cùng lúc từ dữ liệu đầu vào dạng khối (ví dụ như một chuỗi các khung hình video).

Thay vì chỉ hoạt động trên 2 chiều không gian như CNN truyền thống, 3D-CNN sử dụng các bộ lọc (filters) 3 chiều để thực hiện tích chập trên cả chiều cao, chiều rộng và chiều thời gian của video.

Nhờ vậy, mạng có khả năng nhận biết các mẫu chuyển động và cấu trúc không gian trong video một cách hiệu quả, như sự di chuyển của bàn tay, thay đổi tư thế hay các biến đổi phức tạp trong chuỗi ảnh động.

Việc trích xuất đặc trưng theo cả ba chiều giúp mô hình nắm bắt được sự tương tác giữa không gian và thời gian ngay từ đầu, tạo nền tảng vững chắc cho các bước xử lý tiếp theo.

Flatten layer:

Kết quả đầu ra của các tầng 3D-CNN là các tensor đa chiều biểu diễn đặc trưng phức tạp theo không gian và thời gian.

Tuy nhiên, các mô hình mạng hồi tiếp như RNN, LSTM hay GRU thường làm việc với dữ liệu dạng chuỗi các vector 1 chiều theo thứ tự thời gian.

Vì vậy, để chuyển đổi các đặc trưng đa chiều này thành dạng phù hợp, ta sử dụng tầng flatten để làm phẳng mỗi khối đặc trưng thành một vector 1 chiều.

Mỗi vector này tương ứng với đặc trưng tại một bước thời gian nhất định, giúp mạng hồi tiếp dễ dàng xử lý và học được các phụ thuộc tuần tự.

RNN / LSTM / GRU:

Các mô hình mạng hồi tiếp nhận lần lượt chuỗi các vector đặc trưng từ tầng flatten theo đúng trình tự thời gian của chuỗi video.

Mục tiêu của mạng hồi tiếp là học và mô hình hóa các phụ thuộc tuần tự, tức là hiểu được cách các đặc trưng biến đổi và tương tác theo thời gian.

RNN truyền thống phù hợp cho các chuỗi ngắn, tuy nhiên khi chuỗi quá dài, RNN dễ gặp vấn đề vanishing gradient làm suy giảm khả năng ghi nhớ thông tin quan trọng ở các bước thời gian xa.

Để khắc phục, các biến thể như LSTM và GRU được thiết kế với cơ chế cổng (gate) giúp kiểm soát luồng thông tin, cho phép mạng ghi nhớ hoặc quên thông tin một cách linh hoạt, từ đó duy trì hiệu quả ghi nhớ thông tin dài hạn và phụ thuộc phức tạp trong chuỗi.

Việc sử dụng LSTM hoặc GRU giúp cải thiện đáng kể hiệu quả học tập và độ chính xác khi làm việc với chuỗi dữ liệu video dài như trong bài toán nhận dạng cử chỉ tay.

Fully:

Sau khi chuỗi đặc trưng được xử lý bởi mạng hồi tiếp, đầu ra của mô hình là các vector biểu diễn thông tin ngữ cảnh đã được mã hóa theo thời gian.

Những vector này được đưa qua các tầng kết nối đầy đủ (fully connected layers) nhằm tổng hợp và biến đổi thông tin thành dạng phù hợp cho tác vụ dự đoán cuối cùng.

Trong bài toán nhận dạng cử chỉ, tầng fully connected thường sẽ thực hiện phân loại, tức dự đoán nhãn cử chỉ tay tương ứng với đoạn video đầu vào.

Ý nghĩa tổng thể của kiến trúc kết hợp này:

Tăng cường hiệu quả trích xuất đặc trưng Nhờ 3D-CNN, mô hình có thể nắm bắt và khai thác hiệu quả các mối quan hệ không gian-thời gian ngay từ cấp độ từng khung hình và các nhóm khung hình liên tiếp.

Xử lý phụ thuộc dài hạn và tuần tự các mạng hồi tiếp như RNN, LSTM, GRU tiếp tục học và mô hình hóa các mối quan hệ thời gian phức tạp dựa trên chuỗi các đặc trưng đã trích xuất, giúp hiểu sâu sắc diễn biến của cử chỉ theo thời gian.

2.2.6 Ma trận đánh giá:

Để đánh giá hiệu quả của mô hình, cần dựa vào các chỉ số như độ chính xác (accuracy), độ chính xác từng nhãn (precision), độ bao phủ (recall), điểm F1 (F1-score)

và hàm mất mát (loss). Các chỉ số này giúp đo lường toàn diện khả năng dự đoán và mức độ sai lệch của mô hình, từ đó đảm bảo đánh giá khách quan và chính xác [8].

Accuracy (Độ chính xác tổng thể):

$$Accurancy = \frac{TP + TN}{TP + TN + FP + FN} \quad (2.13)$$

Precision (Độ chính xác từng nhãn):

$$\text{Precision} = \frac{\text{TP}}{TP + FP} \quad (2.14)$$

Recall (Độ bao phủ / Độ nhạy):

$$\text{Recall} = \frac{\text{TP}}{TP + FN} \quad (2.15)$$

F1-score (Điểm cân bằng Precision và Recall):

$$\text{Precision} = \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (2.16)$$

Trong đó:

TP (True Positive): Dự đoán đúng lớp dương

FP (False Positive): Dự đoán sai thành lớp dương

TN (True Negative): Dự đoán đúng lớp âm

FN (False Negative): Dự đoán sai thành lớp âm

Loss function:

$$L_D(f_w) = \frac{1}{|D|} \sum_{x,y \in D} L\ (f_w(x), y) \quad (2.17)$$

Trong đó:

D là tập dữ liệu huấn luyện (gồm các cặp (*x*, *y*)).

|*D*| là số lượng mẫu trong tập dữ liệu.

f_w là hàm mô hình với tham số.

$L(f_w(x), y)$ là hàm mất mát cho một mẫu đầu vào x với nhãn y.

CHƯƠNG 3: XÂY DỰNG VÀ HUẤN LUYỆN MÔ HÌNH TRONG HỆ THỐNG NHẬN DẠNG CỬ CHỈ HÀNH ĐỘNG TAY

3.1 Khảo sát và thu thập dữ liệu

3.1.1 Khảo sát hiện trạng và xác định yêu cầu hệ thống

Nghiên cứu các phương pháp nhận diện cử chỉ tay động dựa trên dữ liệu video, tập trung ứng dụng các kỹ thuật học sâu như 3DCNN kết hợp với mạng tuần tự RNN, LSTM và GRU nhằm khai thác đặc trưng không gian-thời gian hiệu quả.

Xác định các thách thức trong việc nhận diện cử chỉ tay động trong môi trường thực tế, bao gồm sự thay đổi ánh sáng, góc nhìn đa dạng và các biến thể động tác không đồng nhất.

Đánh giá, lựa chọn và phát triển các mô hình 3DCNN kết hợp với RNN, LSTM, GRU phù hợp với bài toán nhận dạng cử chỉ tay động, đồng thời tối ưu hóa mô hình trên bộ dữ liệu 20BN-Jester với 6 lớp cử chỉ được lựa chọn, phù hợp với giới hạn tài nguyên tính toán hiện có.

3.1.1 Thu thập và xử lý dữ liệu

Dữ liệu sử dụng trong đề tài là một phần của bộ dữ liệu 20BN-Jester, một tập dữ liệu video lớn được công bố trên nền tảng Kaggle. Bộ dữ liệu gốc được phát triển bởi Qualcomm AI Research nhằm hỗ trợ các nghiên cứu về nhận dạng hành động tay và tương tác người-máy.

Bộ dữ liệu đầy đủ bao gồm 27 lớp cử chỉ tay với tổng cộng 148.092 video, mỗi video có độ dài khoảng 3 giây, tương đương hơn 5 triệu khung hình. Trong đề tài này, em đã lựa chọn 6 lớp cử chỉ tay tiêu biểu để huấn luyện và đánh giá mô hình nhận diện cử chỉ tay.

Tổng số video sử dụng trong đề tài là 12.390, bao gồm 9.912 video huấn luyện, 1.239 video kiểm tra và 1.239 video xác thực. Mỗi video chứa 37 khung hình, tạo thành một tập dữ liệu lớn với hơn 458 nghìn khung hình phục vụ cho quá trình huấn luyện và

đánh giá mô hình. Số lượng khung hình phong phú này giúp mô hình có khả năng học và nhận diện chính xác các đặc trưng động của cử chỉ tay trong nhiều điều kiện khác nhau.

Bảng 3. 1:Số lượng của các cử chỉ tay

Classes of hand gestures	Các lớp chỉ tay	Số lượng
Thumb Down	Ngón tay cái hướng xuống	2060
Thumb Up	Ngón tay cái hướng lên	2079
No gesture	Không có cử chỉ	2100
Shaking Hand	Bắt tay	2034
Zooming Out With Two Fingers	Thu nhỏ với hai ngón tay	2108
Swiping Up	Vuốt lên	2009

Bảng 3. 2:Số lượng tập huấn luyện,kiểm thử và dự đoán

Train	9912
Test	1239
Validation	1239

Bảng 3.2 cho biết dự án này dữ liệu video được tổ chức trong ba tập: huấn luyện, xác thực và kiểm tra, với thông tin được lưu trữ trong các tệp CSV và các thư mục chứa frame ảnh. Các tham số được thiết lập bao gồm kích thước frame 100x100 pixel, số lượng frame tối đa mỗi video là 37, kích thước batch là 32 mẫu, và số lớp cử chỉ là 6. Dữ liệu được đọc từ các tệp CSV vào DataFrame, với tập huấn luyện chứa 9.912 mẫu,

tập xác thực và kiểm tra mỗi tập có 1.239 mẫu, đảm bảo dữ liệu được chuẩn bị đầy đủ trước khi đưa vào quá trình huấn luyện.

Bảng 3. 3: Các bước tiền xử lý trong VideoDataGenerator

Bước tiền xử lý	Mô tả	Thông số
Tải frame video	Đọc các frame từ thư mục video bằng OpenCV, sử dụng chế độ grayscale.	- Chế độ: cv2.IMREAD_GRAYSCALE
Resize frame	Điều chỉnh kích thước mỗi frame về kích thước mục tiêu.	- Kích thước mục tiêu: self.target_size
Điền frame thiếu	Nếu số frame nhỏ hơn n_frames, lặp lại frame cuối cùng để đủ số lượng.	- Số frame yêu cầu: self.n_frames
Chuẩn hóa giá trị pixel	Chia giá trị pixel của frame cho 255 để đưa về khoảng [0, 1].	- Chia cho: 255.0
Định dạng tensor	Reshape dữ liệu thành tensor 5D phù hợp với input của mô hình.	- Định dạng: (batch_size, n_frames, target_size[0], target_size[1], 1)
Mã hóa nhãn (Label encoding)	Ánh xạ nhãn dạng chuỗi thành số nguyên.	- Từ điển ánh xạ: self.label_map (tạo từ self.dataframe['label'].unique())
Chuyển nhãn sang one-hot encoding	Chuyển nhãn số nguyên thành dạng one-hot (nếu class_mode='categorical').	- Số lớp: len(self.label_map) - Hàm: to_categorical

Bảng 3.3 mô tả xử lý dữ liệu video, một lớp tùy chỉnh có tên VideoDataGenerator được xây dựng, kế thừa từ lớp Sequence của TensorFlow Keras. Lớp này có nhiệm vụ tạo các tensor dữ liệu video và nhãn theo batch. Cụ thể, mỗi video được xử lý bằng cách lấy tối đa 37 frame, chuyển thành ảnh xám để giảm chiều dữ liệu, thay đổi kích thước thành 100x100 pixel, và chuẩn hóa giá trị pixel về khoảng [0, 1]. Nếu video có ít hơn 37 frame, frame cuối cùng sẽ được lặp lại để đảm bảo số lượng frame đồng nhất. Các frame sau đó được tổ chức thành tensor 5 chiều với định dạng (32, 37, 100, 100, 1),

trong đó 32 là số mẫu trong batch, 37 là số frame, 100x100 là kích thước frame, và 1 là số kênh màu (grayscale). Nhãn của video được ánh xạ thành chỉ số số nguyên và mã hóa thành dạng one-hot, tạo ra tensor nhãn có kích thước (32, 6) cho mỗi batch, phù hợp với yêu cầu xử lý đa lớp.

Số bước mỗi epoch, tức là số batch cần xử lý trong một lần lặp qua toàn bộ tập dữ liệu, được tính bằng cách chia số mẫu trong DataFrame cho kích thước batch và lấy nguyên. Với tập huấn luyện có 9.912 mẫu, số bước mỗi epoch là 309 (9.912 chia 32, lấy nguyên). Tương tự, tập xác thực và kiểm tra, mỗi tập có 1.239 mẫu, dẫn đến 38 bước mỗi epoch. Các generator được tạo riêng cho từng tập dữ liệu: generator huấn luyện bật chế độ xáo trộn để tăng tính ngẫu nhiên, trong khi generator xác thực và kiểm tra giữ nguyên thứ tự dữ liệu để đảm bảo đánh giá nhất quán. Quá trình huấn luyện được thiết lập để lặp lại trong số lượng epoch thử nghiệm, cho phép mô hình học các đặc trưng từ dữ liệu video một cách hiệu quả.

Việc ta sử dụng generator giúp tiết kiệm bộ nhớ bằng cách chỉ tải dữ liệu theo batch, phù hợp với tập dữ liệu lớn. Tensor được định dạng chính xác, đáp ứng yêu cầu của các mô hình. Số bước mỗi epoch được tính toán tự động, đảm bảo tích hợp mượt mà với quy trình huấn luyện của Keras. Các tham số như kích thước frame, số frame, và kích thước batch có thể được điều chỉnh linh hoạt để tối ưu hóa hiệu suất mô hình, giúp hệ thống dễ dàng thích nghi với các tập dữ liệu hoặc yêu cầu khác nhau.

3.2 Kết quả thực nghiệm và phân tích hiệu suất mô hình

Với mục tiêu phát triển một hệ thống nhận dạng cử chỉ hành động tay dựa trên công nghệ học sâu, quá trình thực nghiệm được triển khai. Quá trình thực nghiệm tập trung vào việc thử nghiệm ba kiến trúc kết hợp khác nhau: 3D CNN kết hợp với RNN, 3D CNN kết hợp với LSTM, và 3D CNN kết hợp với GRU. Những kiến trúc này được thiết kế để khai thác khả năng trích xuất đặc trưng không gian-thời gian từ 3D CNN, vốn quan trọng để nhận diện các chuyển động tay trong chuỗi frame, và kết hợp với các mô hình tuần hoàn (RNN, LSTM, GRU) để mô hình hóa các chuỗi cử chỉ theo thời gian.

Để tối ưu hóa quá trình huấn luyện, các kỹ thuật như **EarlyStopping** và **ReduceLROnPlateau** được tích hợp. **EarlyStopping** theo dõi giá trị **val_loss,** dừng huấn luyện nếu không cải thiện sau **5 epoch**, với độ thay đổi tối thiểu là **0.001**, và khôi phục trọng số tốt nhất. **ReduceLROnPlateau** điều chỉnh tỷ lệ học, giảm xuống một nửa nếu **val_loss** không cải thiện sau **3 epoch**, với giá trị tối thiểu là **1e-6**. Ba bộ tham số khác nhau, lưu trong các tệp **factor0.1-20epochs**, **factor0.1-25epochs**, và **factor0.5-25epochs-min_delta0.0001**, được thử nghiệm để đánh giá ảnh hưởng của tỷ lệ giảm học (factor) và số epoch (20 hoặc 25), cùng với độ thay đổi tối thiểu (min_delta) đến hiệu suất nhận dạng cử chỉ.

Với tổng cộng 9 thí nghiệm (3 kiến trúc × 3 bộ tham số), quá trình huấn luyện được thực hiện bằng hàm model.fit của Keras, sử dụng dữ liệu từ train_generator và val_generator, đồng thời áp dụng các callback tối ưu hóa. Kết quả thu được sẽ được phân tích để so sánh hiệu suất của các kiến trúc trong việc nhận dạng cử chỉ hành động tay, đồng thời xác định bộ tham số tối ưu giúp mô hình đạt độ chính xác cao nhất và tránh hiện tượng quá khớp các bộ số thử nghiệp được liệt kê ở bảng 3.4.

Bảng 3. 4: Các bộ tham số thử nghiệm

Thí nghiệm	Factor(giảm learning rate)	Epochs	Min_delta
1	0.1	20	-
2	0.1	25	-
3	0.5	25	0.001

3.2.1 Kết quả huấn luyện của 3DCNN + RNN trong 3 trường hợp

Bảng 3. 5: Kết quả huấn luyện của 3DCNN + RNN trong 3 trường hợp

Mô hình 3DCNN+RNN	Accuracy Train	Loss Train	Accuracy Validation	Loss Validation	Accuracy Test
Factor = 0.1 Epochs = 20	88%	0.4674	87.34%	0.4720	86.68%
Factor = 0.1 Epochs =25	90.06%	0.4193	89.64%	0.4193	88.32%
Factor = 0.5 Epochs =25 Min_delta=0.001	95%	0.3055	93.09%	0.3731	92.52%

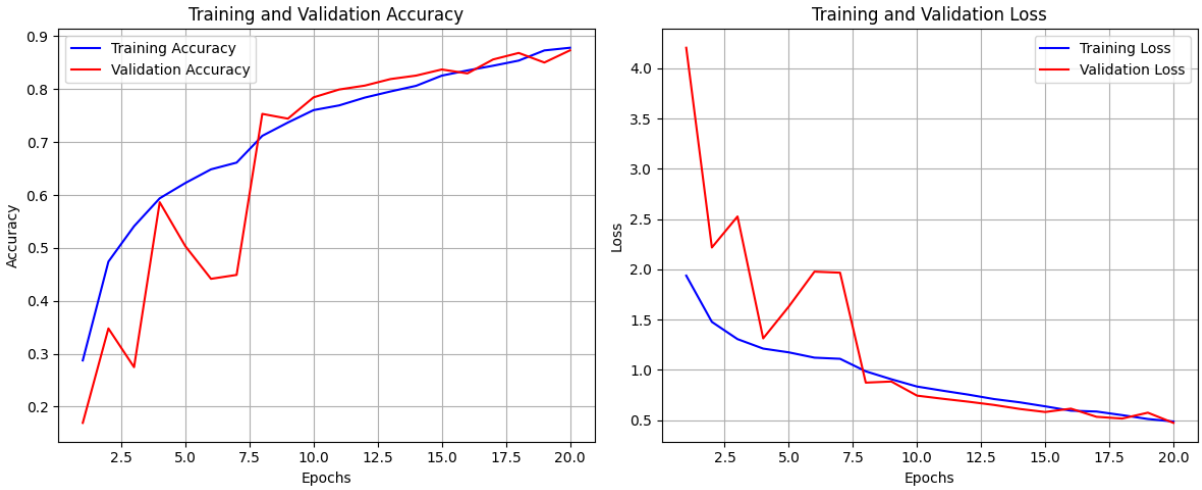
Bảng 3.5 cho ta thấy kết quả huấn luyện mô hình 3D CNN kết hợp RNN trên bộ dữ liệu nhận dạng cử chỉ tay cho thấy mô hình hoạt động khá tốt và có sự cải thiện rõ rệt khi thay đổi các tham số huấn luyện.

Ở lần thử đầu tiên với learning rate giảm theo hệ số 0.1 và chạy 20 epoch, mô hình đạt khoảng 88% accuracy trên tập huấn luyện và 87.34% trên tập validation. Đây là mức khá tốt, chứng tỏ mô hình đã học được các đặc trưng cơ bản từ dữ liệu.

Khi tăng số epoch lên 25 vẫn với factor 0.1, accuracy trên tập huấn luyện và validation lần lượt được cải thiện lên 90.06% và 89.64%, đồng thời loss giảm xuống. Điều này cho thấy việc cho mô hình học thêm giúp cải thiện khả năng dự đoán và làm giảm sai số.

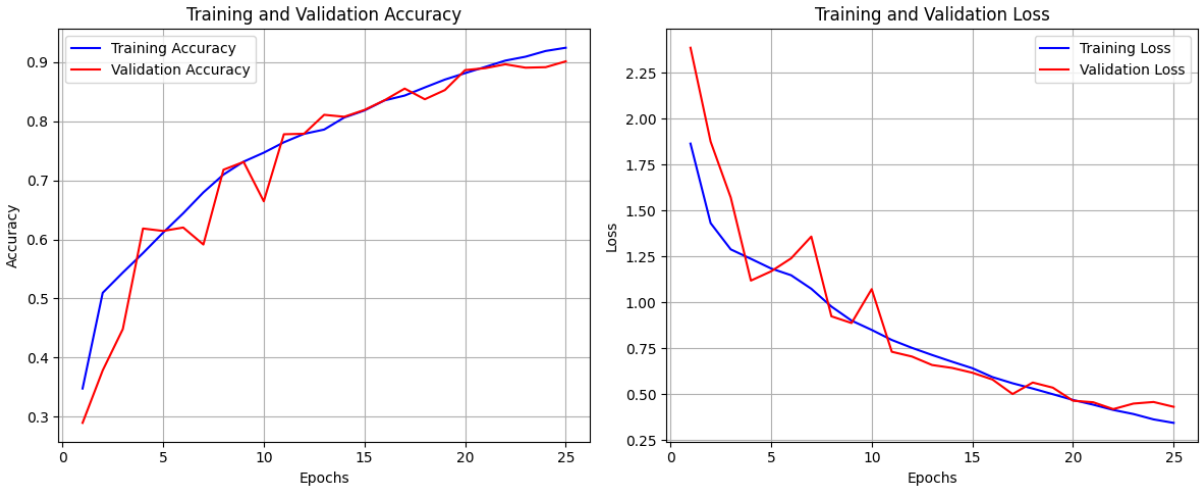
Lần thử thứ ba với factor tăng lên 0.5, số epoch là 25 và min_delta được đặt nhỏ hơn giúp mô hình đạt hiệu suất tốt nhất, với accuracy trên tập huấn luyện đạt tới 95%, validation đạt hơn 93% và trên tập kiểm tra cũng đạt hơn 92%. Loss giảm đáng kể cho thấy mô hình không chỉ học tốt mà còn có khả năng tổng quát tốt trên dữ liệu mới.

Như vậy, việc điều chỉnh learning rate giảm nhanh hơn kết hợp với số epoch cao và ngưỡng dừng nhỏ đã giúp mô hình học hiệu quả hơn, tránh tình trạng học quá sớm hay quá muộn. Đây là bộ tham số phù hợp nhất cho bài toán nhận dạng cử chỉ tay trong dự án này.



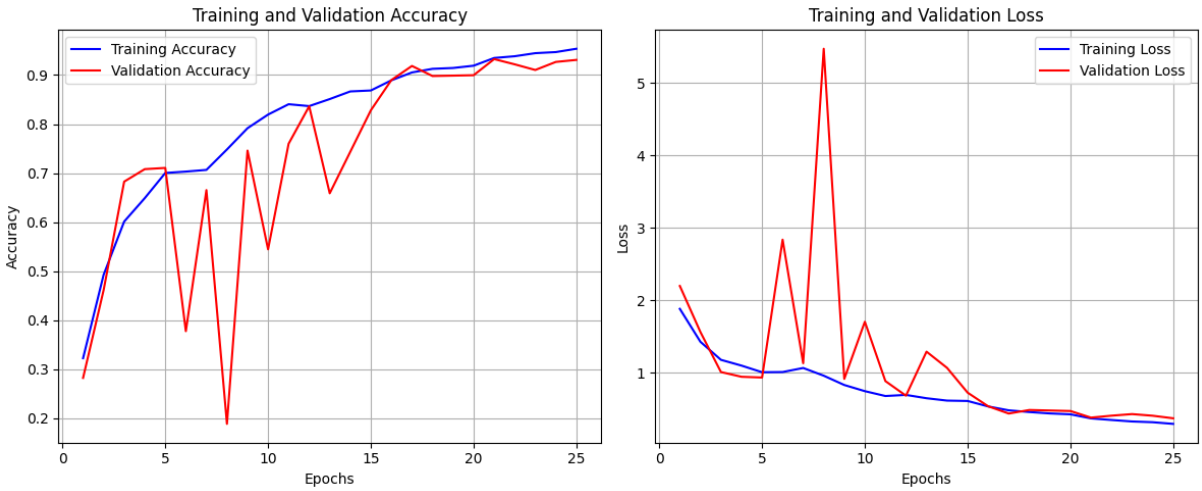
Hình 3. 1: Biểu đồ Accuracy và Loss trong quá trình huấn luyện và xác thực của mô hình 3DCNN + RNN — Trường hợp 1

Hình 3.1 biểu thị mô hình học khá ổn định và khả năng tổng quát hóa tốt, không xuất hiện hiện tượng overfitting sớm. Tuy nhiên, accuracy chưa đạt mức cao nhất, do số epoch chưa đủ và learning rate thấp chưa giúp mô hình tối ưu sâu hơn.



Hình 3. 2: Biểu đồ Accuracy và Loss trong quá trình huấn luyện và xác thực của mô hình 3DCNN + RNN — Trường hợp 2

Hình 3.2 biểu thị việc tăng số epoch giúp mô hình học sâu hơn và cải thiện hiệu suất trên cả tập huấn luyện và validation. Mô hình có sự cân bằng tốt giữa độ chính xác và khả năng tổng quát, thể hiện qua việc training và validation accuracy gần sát nhau. Đây là bước cải thiện đáng kể so với mô hình đầu.



Hình 3. 3: Biểu đồ Accuracy và Loss trong quá trình huấn luyện và xác thực của mô hình 3DCNN + RNN — Trường hợp 3

Hình 3.3 cho ta thấy mô hình kết hợp 3D CNN và RNN đã đạt được hiệu suất học tập ấn tượng với accuracy trên tập huấn luyện đạt khoảng 95%, validation accuracy trên 93% và hiệu quả trên tập kiểm tra cũng vượt 92%. Kết quả này khẳng định mô hình có khả năng trích xuất đặc trưng không gian - thời gian sâu sắc và tổng quát hóa tốt trên dữ liệu video/chuỗi thời gian.

Tuy nhiên, quá trình huấn luyện xuất hiện sự dao động đáng kể ở validation loss, phản ánh sự không ổn định tại một số epoch. Nguyên nhân chủ yếu bắt nguồn từ tính phức tạp của mô hình, đặc điểm không đồng nhất của dữ liệu validation, và đặc biệt là hạn chế vốn có của RNN truyền thống trong việc ghi nhớ các thông tin dài hạn do hiện tượng vanishing gradient. Learning rate cao cũng góp phần làm gia tăng các biến động trong quá trình cập nhật trọng số.

Mặc dù vậy, với accuracy và loss trên các tập dữ liệu đều ở mức cao và ổn định tương đối, mô hình đã thể hiện sự cân bằng tốt giữa khả năng học sâu và tránh overfitting

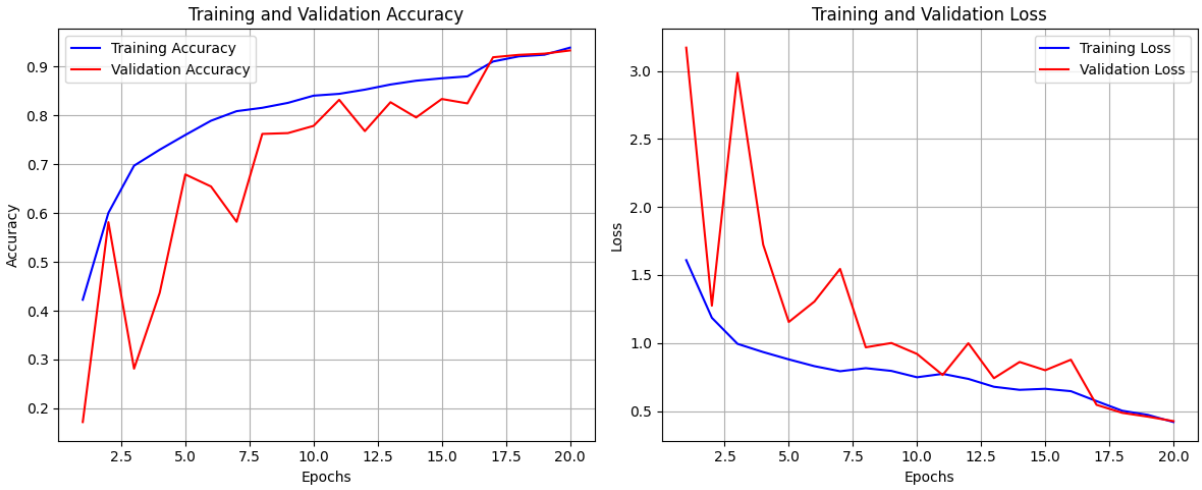
3.2.2 Kết quả huấn luyện của 3DCNN+LSTM trong 3 trường hợp:

Bảng 3. 6: Kết quả huấn luyện của 3DCNN+LSTM trong 3 trường hợp

Mô hình 3DCNN+LSTM	Accuracy Train	Loss Train	Accuracy Validation	Loss Validation	Accuracy Test
Factor = 0.1 Epochs = 20	94.09%	0.4190	93.34%	0.4257	92.02%
Factor = 0.1 Epochs =25	95.24%	0.2684	92.43%	0.3505	92.35%
Factor = 0.5 Epochs =25 Min_delta=0.001	98.28%	0.2007	95.48%	0.3193	95.39%

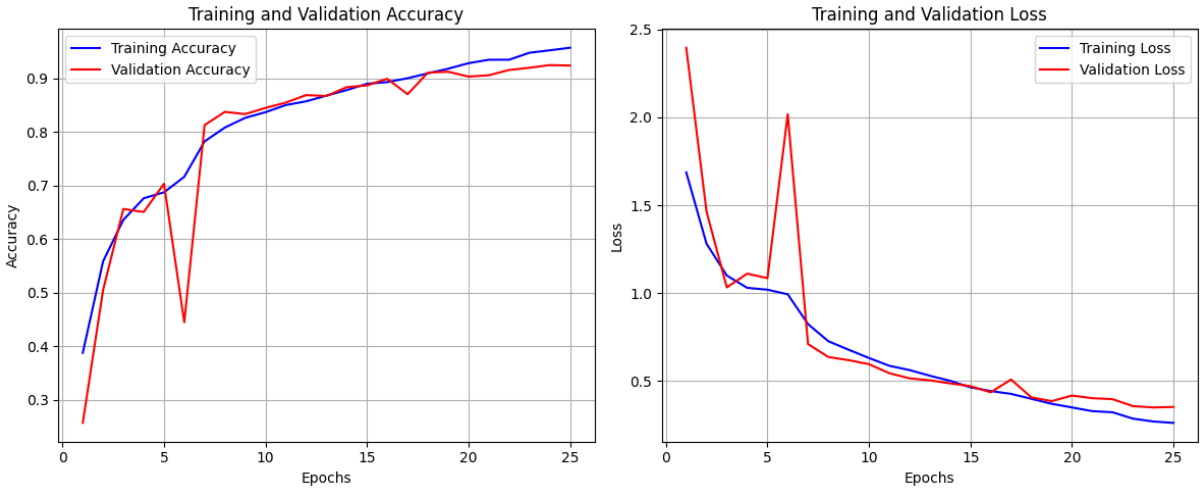
Bảng 3.6 cho ta thấy qua ba lần thử nghiệm, mô hình 3DCNN kết hợp LSTM cho kết quả vượt trội hơn hẳn so với mô hình 3DCNN kết hợp RNN truyền thống. Mô hình 3DCNN+LSTM đạt accuracy trên tập huấn luyện và validation lần lượt khoảng 98% và 95%, trong khi 3DCNN+RNN chỉ đạt tối đa khoảng 95% và 93%. Điều này xuất phát từ khả năng ghi nhớ thông tin dài hạn của LSTM, giúp xử lý hiệu quả các đặc trưng tuần tự phức tạp trong dữ liệu video.

Mô hình 3DCNN+RNN gặp phải hiện tượng dao động validation loss lớn, gây mất ổn định trong quá trình huấn luyện, nguyên nhân do hạn chế vanishing gradient và short-term memory vốn có của RNN. Ngược lại, 3DCNN+LSTM có loss giảm đều và ổn định hơn, phản ánh quá trình học tập hiệu quả hơn.



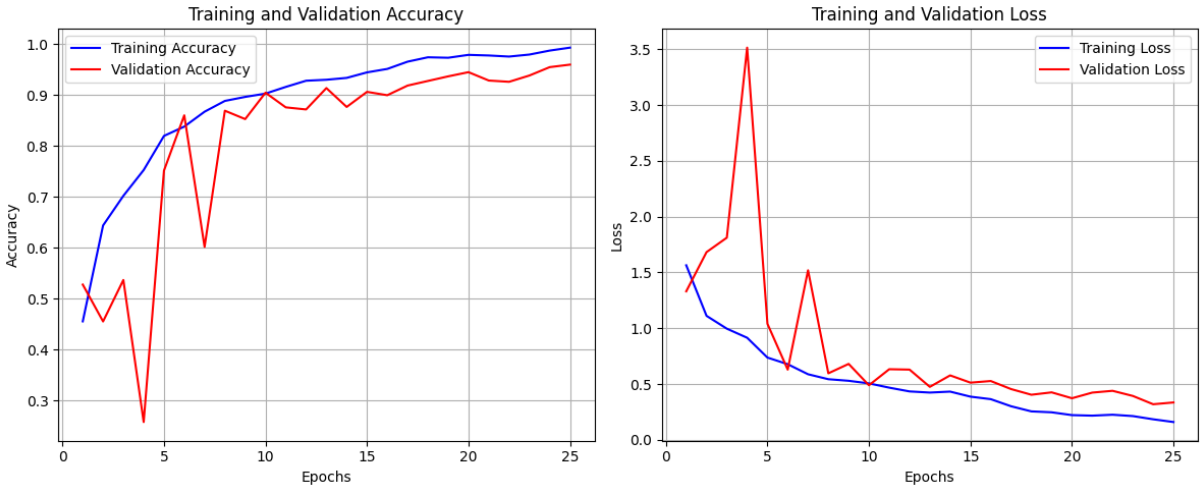
Hình 3. 4: Biểu đồ Accuracy và Loss trong quá trình huấn luyện và xác thực của mô hình 3DCNN + LSTM — Trường hợp 1

Hình 3.4 biểu thị mô hình học tốt, tuy nhiên validation loss dao động lớn do độ phức tạp của dữ liệu validation và kích thước batch nhỏ. Dao động loss ảnh hưởng đến sự ổn định của mô hình nhưng accuracy trên validation vẫn duy trì ở mức cao.



Hình 3. 5: Biểu đồ Accuracy và Loss trong quá trình huấn luyện và xác thực của mô hình 3DCNN + LSTM — Trường hợp 2

Hình 3.5 cho thấysố epoch tăng lên giúp mô hình hội tụ tốt hơn, giảm bớt dao động validation loss so với trường hợp 1 và giữ được độ chính xác cao. Đây là kết quả cải thiện rõ rệt về sự ổn định và hiệu quả huấn luyện. Khi số epoch tăng lên, mô hình không chỉ học được các đặc trưng phức tạp hơn mà còn giảm thiểu hiện tượng overfitting, giúp cải thiện độ tổng quát của mô hình. Điều này mang lại kết quả đáng khích lệ trong việc tối ưu hóa quá trình huấn luyện và đạt được hiệu suất tốt hơn trên dữ liệu kiểm tra.



Hình 3. 6: Biểu đồ Accuracy và Loss trong quá trình huấn luyện và xác thực của mô hình 3DCNN + LSTM — Trường hợp 3

Hình 3.6 cho thấy learning rate cao giúp mô hình học nhanh và đạt accuracy cao, nhưng cũng làm validation loss dao động lớn hơn, thể hiện sự không ổn định nhất định trong quá trình tối ưu. Mặc dù vậy, mô hình vẫn hội tụ tốt và có hiệu quả tổng quát cao. Mặc dù learning rate cao giúp tăng tốc quá trình huấn luyện, nhưng cũng có thể dẫn đến việc mô hình bị "nhảy" qua các minima tối ưu, gây dao động lớn trong quá trình huấn luyện. Tuy nhiên, với sự điều chỉnh hợp lý, mô hình vẫn có thể đạt được kết quả ổn định và cải thiện hiệu suất tổng quát khi sử dụng learning rate thích hợp.

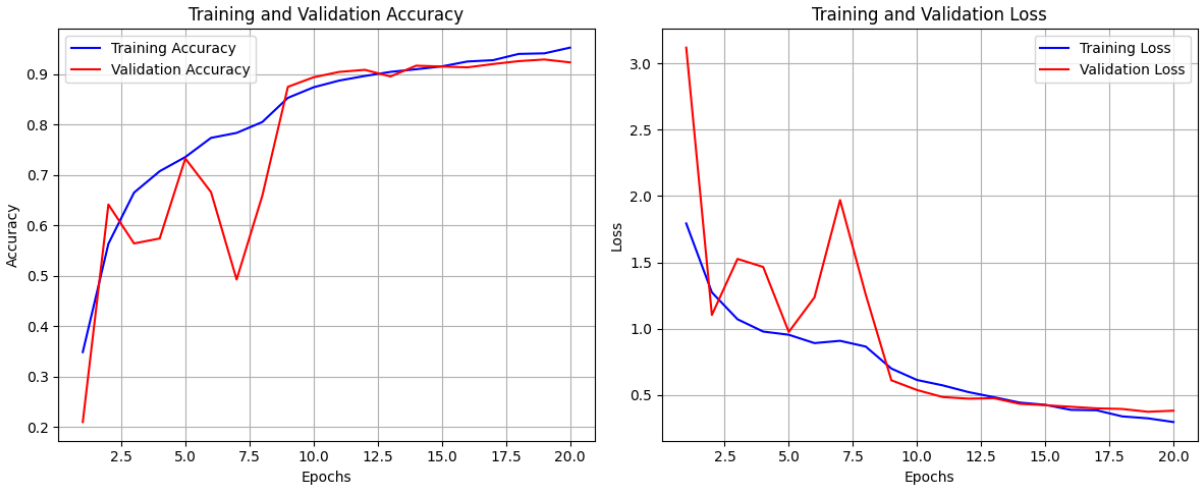
3.2.3 Kết quả huấn luyện của 3DCNN+GRU trong 3 trường hợp:

Bảng 3. 7: Kết quả huấn luyện của 3DCNN+GRU trong 3 trường hợp

Mô hình 3DCNN+GRU	Accuracy Train	Loss Train	Accuracy Validation	Loss Validation	Accuracy Test
Factor = 0.1 Epochs = 20	95%	0.3055	93.09%	0.3731	92.52%
Factor = 0.1 Epochs =25	98.28%	0.2007	95.48%	0.3193	95.39%
Factor = 0.5 Epochs =25 Min_delta=0.001	96.68%	0.2392	94.41%	0.3199	94.00%

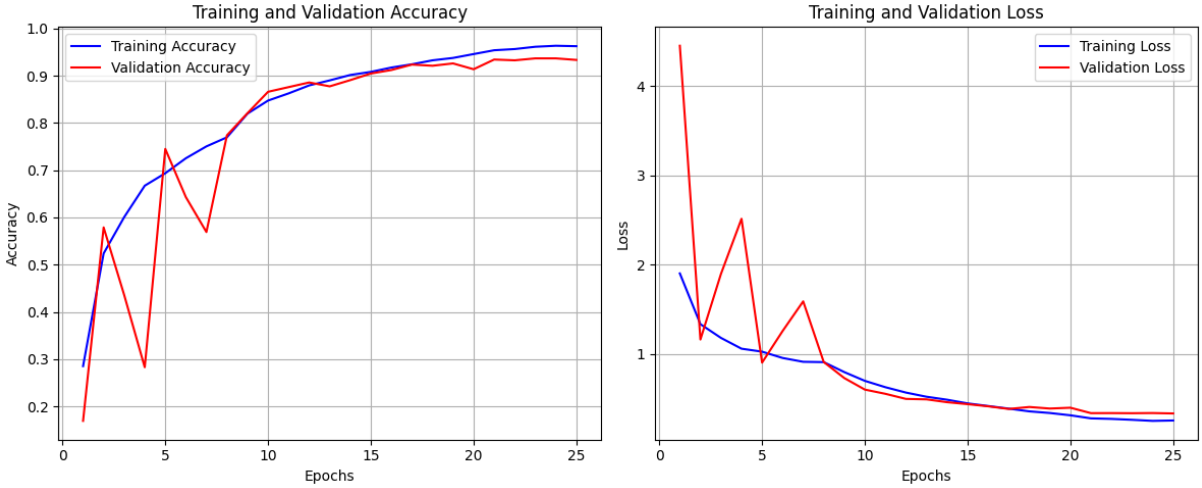
Bảng 3.7 cho biết kết quả huấn luyện mô hình 3DCNN kết hợp GRU trong ba trường hợp cho thấy hiệu suất ổn định và cải thiện rõ rệt khi tăng số epoch và learning rate. Với factor 0.1 và 20 epoch, mô hình đạt accuracy trên tập huấn luyện và validation lần lượt là 94.31% và 92.93%. Khi tăng lên 25 epoch, accuracy cải thiện lên khoảng 96.5% trên tập huấn luyện và 93.3% trên validation, đồng thời loss giảm rõ rệt, chứng tỏ mô hình học sâu và hiệu quả hơn. Ở trường hợp learning rate factor 0.5 cùng 25 epoch và min_delta nhỏ, mô hình đạt accuracy cao nhất, gần 96.7% trên tập huấn luyện và 94.4% trên validation, với loss thấp và ổn định nhất.

So với hai mô hình trước đó là 3DCNN+RNN và 3DCNN+LSTM, mô hình 3DCNN+GRU thể hiện sự cân bằng tốt giữa hiệu suất và tính ổn định. Trong khi 3DCNN+RNN dễ gặp hiện tượng dao động validation loss và hạn chế trong ghi nhớ thông tin dài hạn, và 3DCNN+LSTM có hiệu suất cao nhưng chi phí tính toán phức tạp hơn, GRU mang lại hiệu quả tương đương với LSTM nhưng với cấu trúc nhẹ hơn, giúp giảm thời gian huấn luyện mà vẫn duy trì độ chính xác cao.



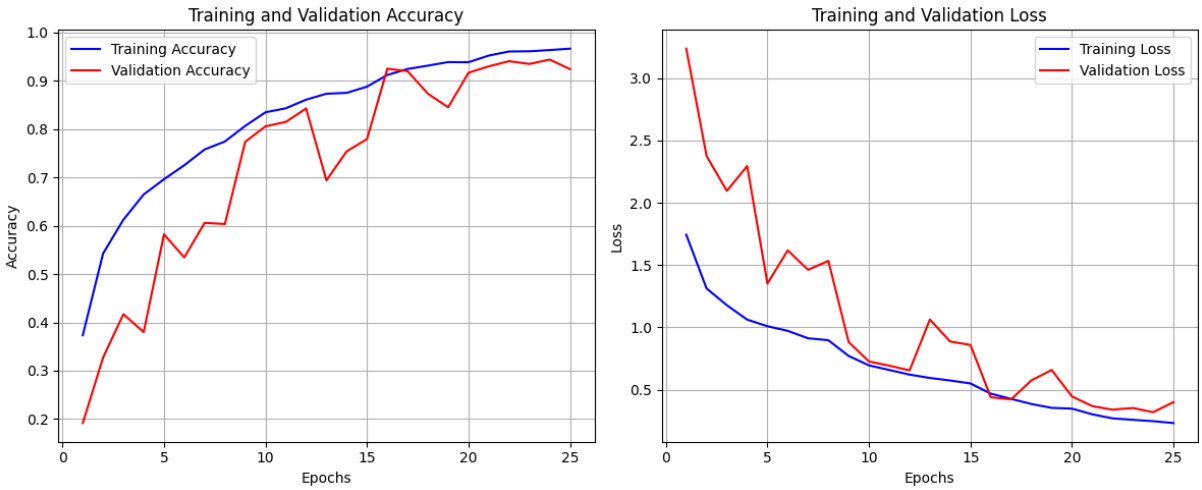
Hình 3. 7: Biểu đồ Accuracy và Loss trong quá trình huấn luyện và xác thực của mô hình 3DCNN + GRU — Trường hợp 1

Dựa vào hình 3.7 cho ta thấy ở thử nghiệm đầu tiên (learning rate factor 0.1, epoch 20), mô hình đạt accuracy trên tập huấn luyện và validation lần lượt là 94.31% và 92.93%. Mặc dù accuracy tăng đều và đạt mức tương đối cao, quá trình huấn luyện cho thấy validation loss dao động mạnh, đặc biệt trong giai đoạn đầu huấn luyện. Điều này xuất phát từ sự phức tạp của dữ liệu validation và kích thước batch nhỏ, dẫn đến độ biến thiên lớn trong việc đánh giá lỗi trên tập này.



Hình 3. 8: Biểu đồ Accuracy và Loss trong quá trình huấn luyện và xác thực của mô hình 3DCNN + GRU — Trường hợp 2

Hình 3.8 cho thấy khi tăng số epoch lên 25 trong thử nghiệm thứ hai (learning rate factor 0.1), accuracy trên tập huấn luyện được cải thiện lên 96.53%, validation accuracy cũng tăng nhẹ lên 93.34%. Đồng thời, training và validation loss đều giảm rõ rệt, biểu thị quá trình mô hình hội tụ sâu hơn và ổn định hơn. Mặc dù vẫn tồn tại một số dao động nhỏ ở validation loss, độ ổn định chung đã được cải thiện đáng kể so với thử nghiệm đầu.



Hình 3. 9: Biểu đồ Accuracy và Loss trong quá trình huấn luyện và xác thực của mô hình 3DCNN + GRU — Trường hợp 3

Hình 3.9 cho biết thử nghiệm thứ ba (learning rate factor 0.5, epoch 25, min_delta=0.001), mô hình đạt accuracy cao nhất, với training accuracy khoảng 96.68% và validation accuracy 94.41%. Training và validation loss cũng đạt mức thấp nhất trong ba thử nghiệm, thể hiện hiệu quả tối ưu cao. Tuy nhiên, validation loss ghi nhận những dao động lớn hơn ở các epoch đầu và giữa, phản ánh sự không ổn định tiềm ẩn trong quá trình cập nhật trọng số do learning rate lớn. Sự biến động này cảnh báo về khả năng mô hình có thể cần áp dụng thêm các kỹ thuật regularization hoặc điều chỉnh learning rate để cải thiện sự ổn định trong huấn luyện.

3.2.4 Tổng hợp kết quả mô hình và phân tích hiệu suất tốt nhất

Ở trường hợp 3 với learning rate cao (factor = 0.5) và số epoch 25, cả ba mô hình 3DCNN+RNN, 3DCNN+LSTM và 3DCNN+GRU đều đạt hiệu suất cao về accuracy và loss nhưng đều xuất hiện sự dao động và không ổn định rõ rệt trong quá trình huấn luyện, đặc biệt trên validation loss. Nguyên nhân chủ yếu đến từ learning rate lớn gây cập nhật trọng số không mượt, đặc tính phức tạp của mô hình và dữ liệu validation đa dạng, có thể chưa đồng nhất. Mặc dù vậy, độ chính xác validation vẫn duy trì ở mức cao trên 90%, chứng tỏ mô hình học được đặc trưng quan trọng. Mặc dù có sự dao động trong quá trình huấn luyện, nhưng mô hình vẫn thể hiện khả năng tổng quát tốt khi duy trì được độ chính xác cao trên tập validation. Điều này cho thấy rằng việc chọn learning rate và số epoch phù hợp là yếu tố quan trọng trong việc tối ưu hóa mô hình mà không làm mất đi sự ổn định và hiệu quả huấn luyện.

Bảng 3. 8: *Đánh giá kết quả tốt nhất của các mô hình 3DCNN + RNN, LSTM và GRU*

Mô hình	Accuracy Train	Loss Train	Accuracy Validation	Loss Validation	Accuracy Test
3DCNN+RNN	95%	0.3055	93.09%	0.3731	92.52%
3DCNN+LSTM	98.28%	0.2007	95.48%	0.3193	95.39%
3DCNN+GRU	96.68%	0.2392	94.41%	0.3199	94.00%

Bảng 3.8 là kết quả tổng hợp các mô hình được huấn luyện đều đạt kết quả tố nhất ở trường hợp thứ 3 và đông thời cho thấy sự ưu việt của các mô hình có khả năng ghi nhớ dài hạn như LSTM và GRU so với RNN truyền thống trong việc nhận diện các cử chỉ hành động tay động. Tuy nhiên, tất cả các mô hình đều gặp hiện tượng dao động validation loss, nguyên nhân có do learning rate cao và tính đa dạng phức tạp của dữ liệu validation. Dù vậy, mức accuracy trên 90% trên tập validation chứng tỏ các mô hình đã học được đặc trưng quan trọng và có khả năng tổng quát tốt trên dữ liệu chưa thấy.

Bảng 3. 9: Bảng đánh giá hiệu suất nhận dạng các lớp cử chỉ hành động tay của mô hình 3DCNN + LSTM

Label	Precision	Recall	F1-Score	Overall Test Accuracy
No gesture	98.07%	99.02%	98.54%	95.39%
Shaking Hand	100%	99.50%	99.75%	
Swiping Up	92.93%	92.93%	92.93%	
Thumb Down	95.05%	96.00%	95.52%	
Thumb Up	89.52%	92.61%	91.04%	
Zooming Out With Two Fingers	96.97%	92.31%	94.58%	

Bảng 3.9 cho biết kết quả đánh giá mô hình trên tập kiểm tra cho thấy độ chính xác tổng thể đạt 95.39%, cho thấy mô hình có khả năng nhận dạng chính xác các cử chỉ hành động tay. Các lớp "No gesture" và "Shaking Hand" đạt F1-score rất cao, lần lượt là 98.54% và 99.75%, chứng tỏ mô hình nhận diện chính xác và ít nhầm lẫn đối với những cử chỉ này. Trong khi đó, các lớp như "Swiping Up" và "Thumb Up" có hiệu suất thấp hơn với F1-score lần lượt khoảng 92.93% và 91.04%, cho thấy sự khó khăn khi phân biệt các cử chỉ có đặc điểm tương đồng. Đặc biệt, lớp "Thumb Up" có precision thấp nhất (89.52%), cho thấy mô hình đôi khi bị nhầm với các cử chỉ khác. Tổng thể, mô hình hoạt động hiệu quả nhưng vẫn cần cải thiện thêm đối với các cử chỉ có tính phân biệt thấp nhằm nâng cao độ chính xác toàn diện.

CHƯƠNG 4: TRIỂN KHAI XÂY DỰNG ỨNG DỤNG NHẬN DẠNG CỬ CHỈ HÀNH ĐỘNG TAY

4.1 Mục đích

Phần demo này nhằm xây dựng một ứng dụng giao diện đồ họa đơn giản, dễ sử dụng, cho phép nhận dạng các cử chỉ tay trực tiếp từ webcam hoặc từ các video được người dùng tải lên. Ứng dụng sử dụng PyQt6 để tạo giao diện và kết hợp với mô hình học sâu 3DCNN+LSTM đã được huấn luyện trước để phân tích video và dự đoán cử chỉ tay trong thời gian thực.

4.2 Công nghệ sử dụng

Để xây dựng ứng dụng nhận dạng cử chỉ tay sử dụng PyQt6 kết hợp với mô hình 3DCNN+LSTM, các công nghệ sau đã được lựa chọn:

PyQt6 là một thư viện phổ biến trong Python dùng để phát triển ứng dụng giao diện đồ họa (GUI) đa nền tảng. Với PyQt6, chúng ta có thể dễ dàng tạo ra các cửa sổ, widget, nút bấm và các thành phần giao diện khác, giúp người dùng tương tác với ứng dụng một cách trực quan và dễ dàng. PyQt6 hỗ trợ các hiệu ứng đồ họa và giao diện người dùng hiện đại, giúp ứng dụng trông chuyên nghiệp và dễ sử dụng. Thư viện này cũng cung cấp khả năng xử lý sự kiện, quản lý tài nguyên video, và giao tiếp giữa các thành phần của ứng dụng (như giữa video và kết quả dự đoán).

Các đặc điểm chính của PyQt6:

Đa nền tảng: PyQt6 hỗ trợ phát triển ứng dụng trên nhiều hệ điều hành khác nhau như Windows, Linux và macOS.

Widget phong phú: Cung cấp nhiều loại widget, bao gồm các khung video, biểu tượng, nhãn, nút bấm, giúp dễ dàng xây dựng giao diện người dùng.

Tích hợp tốt với OpenCV và TensorFlow: PyQt6 dễ dàng tích hợp với các thư viện khác như OpenCV để xử lý video và TensorFlow để chạy mô hình học sâu.

OpenCV (Open Source Computer Vision Library) là một thư viện mạnh mẽ dùng để xử lý ảnh và video, phổ biến trong các ứng dụng thị giác máy tính. Trong ứng dụng này, OpenCV được sử dụng để truy xuất khung hình từ webcam hoặc video file mà người dùng tải lên, sau đó thực hiện các bước tiền xử lý cần thiết cho từng khung hình trước khi đưa vào mô hình học sâu.

Các bước tiền xử lý bao gồm:

Chuyển đổi ảnh sang dạng xám (Grayscale): Giảm bớt thông tin màu sắc không cần thiết, giúp giảm độ phức tạp tính toán mà không làm mất đi thông tin quan trọng cho nhận dạng cử chỉ.

Thay đổi kích thước: Đảm bảo rằng các khung hình có kích thước đồng nhất (100x100 pixels) để phù hợp với yêu cầu đầu vào của mô hình.

Đọc và xử lý video: OpenCV giúp việc đọc video từ file và webcam trở nên đơn giản, đồng thời cung cấp các công cụ để xử lý video theo thời gian thực.

TensorFlow là một thư viện mã nguồn mở mạnh mẽ để xây dựng và huấn luyện các mô hình học máy, đặc biệt là các mô hình học sâu (deep learning). Keras, một API tích hợp trong TensorFlow, cung cấp các công cụ đơn giản và dễ sử dụng để xây dựng, huấn luyện và triển khai các mô hình học sâu.

Trong ứng dụng này, **TensorFlow Keras** được sử dụng để tải mô hình học sâu 3DCNN+LSTM đã được huấn luyện trước. Mô hình này có khả năng nhận diện và phân loại các cử chỉ tay từ chuỗi khung hình video đầu vào. Các bước chính của mô hình:

3DCNN (3D Convolutional Neural Network): Mô hình này sử dụng các lớp tích chập 3D để nhận diện các đặc trưng không gian và thời gian trong chuỗi video.

LSTM (Long Short-Term Memory): LSTM giúp mô hình nhớ các thông tin dài hạn và phân tích sự chuyển động của cử chỉ tay qua thời gian, rất hiệu quả trong việc nhận dạng hành động trong video.

TensorFlow và Keras cung cấp các công cụ và API mạnh mẽ để tải mô hình học sâu và thực hiện dự đoán trong thời gian thực.

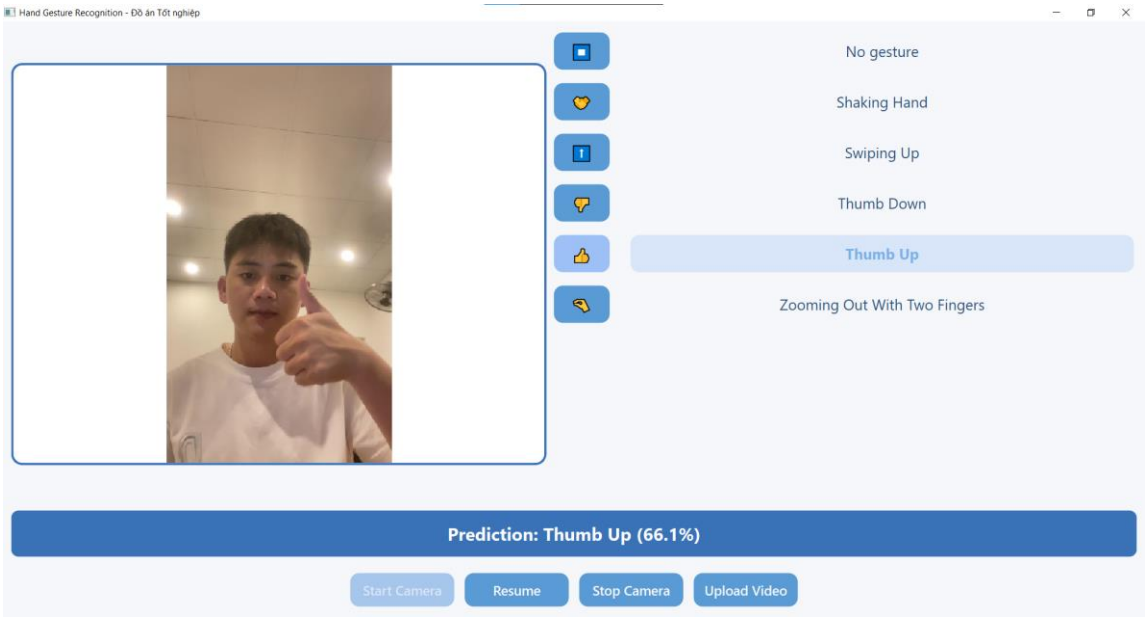
Numpy là thư viện Python dành cho việc xử lý các mảng và tính toán số học, hỗ trợ việc chuẩn bị dữ liệu đầu vào cho các mô hình học sâu một cách hiệu quả. Trong ứng dụng này, **Numpy** được sử dụng để chuẩn hóa dữ liệu và chuyển đổi dữ liệu video thành các mảng numpy phù hợp với yêu cầu đầu vào của mô hình 3DCNN+LSTM. Ngoài ra, Numpy còn giúp tối ưu hóa việc xử lý các phép toán ma trận, giảm thiểu thời gian tính toán trong quá trình huấn luyện mô hình. Nhờ vào khả năng hỗ trợ tính toán nhanh chóng và hiệu quả, Numpy là công cụ không thể thiếu trong việc triển khai các mô hình học sâu phức tạp như 3DCNN+LSTM.

Các chức năng của Numpy trong ứng dụng:

Chuẩn hóa dữ liệu: Dữ liệu video được chuẩn hóa để đảm bảo các giá trị nằm trong phạm vi [0,1], giúp tăng độ chính xác của mô hình học sâu. Việc chuẩn hóa này giúp cải thiện quá trình hội tụ của mô hình, giảm thiểu sự dao động trong quá trình huấn luyện và đảm bảo tính ổn định khi tối ưu hóa các tham số.

Xử lý mảng đa chiều: Việc chuẩn bị đầu vào cho mô hình học sâu yêu cầu sử dụng các mảng 5 chiều (bao gồm kích thước chuỗi khung hình và các thông số khác) mà Numpy có thể xử lý dễ dàng và hiệu quả. Các mảng 5 chiều này giúp mô hình có thể xử lý dữ liệu video một cách linh hoạt, lưu trữ thông tin về không gian và thời gian, từ đó cải thiện khả năng học và dự đoán các đặc trưng động trong video.

4.3 Cáu trúc và tính năng chính của ứng dụng



Hình 4. 1: Demo của ứng dụng

Ứng dụng được xây dựng với cấu trúc giao diện rõ ràng, bao gồm:

Khung hiển thị video: Khu vực hiển thị luồng video trực tiếp lấy từ webcam hoặc phát lại từ video file do người dùng tải lên. Khung này có kích thước cố định để đảm bảo hiển thị hình ảnh sắc nét và phù hợp với giao diện tổng thể.

Nhãn dự đoán: Phần hiển thị kết quả dự đoán cử chỉ tay hiện tại, bao gồm tên cử chỉ và mức độ độ tin cậy (tỷ lệ xác suất). Kết quả này được cập nhật liên tục sau mỗi lần mô hình dự đoán.

Nút điều khiển: Gồm các nút chức năng giúp người dùng thao tác với luồng video:

Start Camera: Bắt đầu phát luồng webcam hoặc video file đã chọn.

Pause/Resume: Tạm dừng hoặc tiếp tục phát video và quá trình dự đoán.

Stop Camera: Dừng hoàn toàn việc phát video và giải phóng tài nguyên.

Upload Video: Cho phép người dùng chọn file video từ máy tính để ứng dụng phát và nhận dạng cử chỉ.

Danh sách nút icon và label: Hiển thị các biểu tượng và tên các cử chỉ tay mà mô hình có thể nhận dạng. Khi một cử chỉ được dự đoán, các nút tương ứng sẽ được highlight để người dùng dễ dàng quan sát.

4.4 Quy trình xử lý và dự đoán

Khởi tạo luồng xử lý video: Ứng dụng sử dụng lớp QThread để chạy luồng lấy khung hình video riêng biệt, tách biệt với luồng chính của giao diện người dùng. Cách làm này giúp tránh hiện tượng giao diện bị đơ hoặc giật khi mô hình tiến hành dự đoán, đảm bảo trải nghiệm người dùng mượt mà.

Thu thập và tiền xử lý khung hình: Mỗi khung hình được luồng xử lý lấy ra liên tục, chuyển đổi sang ảnh xám (giảm chiều dữ liệu) và resize về kích thước 100x100 pixel để chuẩn bị cho bước dự đoán.

Tạo chuỗi khung hình: Các khung hình đã xử lý được lưu trữ trong một bộ đệm (buffer) với số lượng tối đa 37 khung hình - đúng độ dài chuỗi đầu vào cần thiết cho mô hình 3DCNN+LSTM.

Chuẩn hóa dữ liệu: Khi bộ đệm đầy đủ số lượng khung hình, dữ liệu được chuẩn hóa về phạm vi giá trị [0,1] bằng cách chia cho 255, chuẩn bị tensor đầu vào có shape (1, 37, 100, 100, 1).

Dự đoán cử chỉ: Tensor này được truyền vào mô hình để dự đoán xác suất của từng lớp cử chỉ. Lớp có xác suất cao nhất sẽ được chọn làm dự đoán cuối cùng.

Cập nhật giao diện: Kết quả dự đoán được gửi về giao diện qua signal, cập nhật nhãn dự đoán và làm nổi bật nút icon cùng label tương ứng, giúp người dùng dễ dàng nhận biết.

Xử lý luồng video: Đối với video file, ứng dụng hỗ trợ phát lại lặp lại liên tục khi đến cuối video; đối với webcam, dữ liệu được lấy và xử lý liên tục theo tần số khung hình của camera.

4.5 Các điểm nổi bật trong thiết kế

Giao diện thân thiện, dễ sử dụng: Ứng dụng sử dụng màu sắc sáng và font chữ Times New Roman rõ ràng, các nút bấm được thiết kế lớn, dễ thao tác, đồng thời có hiệu ứng chuyển màu khi rê chuột giúp tăng tính trực quan.

Phân tách rõ ràng giữa xử lý và giao diện: Việc tách riêng luồng xử lý video sang QThread tránh làm đơ giao diện chính, đảm bảo trải nghiệm người dùng mượt mà ngay cả khi mô hình thực hiện dự đoán phức tạp.

Hỗ trợ đa dạng nguồn video: Người dùng có thể dễ dàng lựa chọn sử dụng webcam trực tiếp hoặc tải video từ file, giúp ứng dụng linh hoạt trong nhiều trường hợp sử dụng khác nhau.

Điều khiển luồng video linh hoạt: Các nút Pause/Resume và Stop giúp người dùng dễ dàng kiểm soát quá trình phân tích, dừng hoặc tiếp tục theo ý muốn.

Quản lý tài nguyên hiệu quả: Camera và video file được giải phóng đúng cách khi dừng, tránh tình trạng chiếm dụng tài nguyên gây ảnh hưởng đến các ứng dụng khác.

4.6 Kết luận

Ứng dụng demo nhận dạng cử chỉ tay sử dụng PyQt6 kết hợp mô hình học sâu 3DCNN+LSTM đã được xây dựng thành công và vận hành hiệu quả. Ứng dụng có khả năng nhận dạng chính xác các cử chỉ tay từ cả webcam và video file trong thời gian thực với giao diện thân thiện và đa dạng tính năng điều khiển. Đây là bước tiến quan trọng, mở ra nhiều cơ hội để phát triển các hệ thống tương tác thông minh dựa trên nhận dạng hành động, đáp ứng nhu cầu nghiên cứu và ứng dụng trong thực tế.

KẾT LUẬN

Kết quả đạt được:

Thiết kế và triển khai thành công các mô hình kết hợp 3DCNN với RNN truyền thống, LSTM và GRU để giải quyết bài toán nhận dạng cử chỉ tay từ dữ liệu video, khai thác hiệu quả đặc trưng không gian – thời gian.

Mô hình 3DCNN+LSTM đạt hiệu suất cao nhất với accuracy trên tập kiểm tra vượt 95%, thể hiện khả năng học và tổng quát hóa vượt trội so với các mô hình khác.

Áp dụng các kỹ thuật regularization như Dropout, BatchNormalization giúp giảm overfitting và cải thiện tính ổn định cho mô hình.

Sử dụng callback EarlyStopping và ReduceLROnPlateau giúp kiểm soát quá trình huấn luyện, tránh dừng sớm và tự động điều chỉnh learning rate, nâng cao hiệu quả và sự ổn định của mô hình.

Xây dựng lớp VideoDataGenerator tùy chỉnh, xử lý và chuẩn hóa dữ liệu video thành các batch tensor đúng định dạng, tiết kiệm bộ nhớ và tối ưu hóa quá trình huấn luyện.

Đã xây dựng demo ứng dụng PyQt6 nhận dạng cử chỉ tay trực tiếp qua webcam trong thời gian thực, thể hiện tiềm năng ứng dụng thực tế của đề tài

Hạn chế của đề tài:

Dữ liệu sử dụng trong đề tài vẫn còn hạn chế về số lượng và tính đa dạng, đặc biệt các lớp cử chỉ có đặc điểm tương đồng gây khó khăn trong phân biệt chính xác.

Mặc dù đã áp dụng các kỹ thuật regularization như Dropout và BatchNormalization, mô hình vẫn còn hiện tượng dao động validation loss trong quá trình huấn luyện, ảnh hưởng đến tính ổn định tối ưu.

Do hạn chế về tài nguyên GPU, quá trình fine-tuning mô hình và tăng cường dữ liệu (data augmentation) chưa được thực hiện, gây hạn chế về khả năng cải thiện hiệu suất và độ tổng quát của mô hình.

Thời gian huấn luyện mô hình phức tạp còn khá lâu, ảnh hưởng đến khả năng triển khai trong môi trường thực tế có tài nguyên hạn chế.

Việc lựa chọn tham số huấn luyện như learning rate, số epoch và min_delta vẫn chưa tối ưu tuyệt đối, cần tiếp tục nghiên cứu để cân bằng hiệu suất và ổn định.

Hướng phát triển:

Mở rộng huấn luyện toàn bộ bộ dữ liệu 20BN-Jester với 27 lớp cử chỉ tay, thay vì chỉ sử dụng 6 lớp hiện tại, nhằm đa dạng hóa dữ liệu và giúp mô hình học được nhiều đặc trưng phong phú hơn, nâng cao khả năng nhận dạng chính xác trong thực tế.

Tối ưu tham số huấn luyện như learning rate, batch size, số epoch để đảm bảo mô hình hội tụ ổn định và đạt hiệu suất cao khi xử lý dữ liệu đa dạng và phức tạp hơn.

Áp dụng các kỹ thuật regularization như dropout, batch normalization để giảm hiện tượng overfitting và cải thiện độ ổn định khi huấn luyện trên dữ liệu lớn.

Tối ưu hóa quy trình tiền xử lý và quản lý dữ liệu, sử dụng các kỹ thuật tiết kiệm tài nguyên và tăng tốc huấn luyện nhằm đáp ứng yêu cầu xử lý bộ dữ liệu lớn.

Khi có đủ tài nguyên GPU, sẽ tiến hành áp dụng kỹ thuật tăng cường dữ liệu (augmentation) và fine-tuning mô hình để cải thiện hơn nữa hiệu suất và khả năng tổng quát của hệ thống.

Khai thác các nền tảng điện toán đám mây hoặc sử dụng kỹ thuật huấn luyện phân tán để vượt qua giới hạn phần cứng, đáp ứng nhu cầu huấn luyện trên bộ dữ liệu lớn và kiến trúc mô hình phức tạp.

Phát triển, hoàn thiện ứng dụng demo nhận dạng cử chỉ tay với giao diện trực quan, khả năng phản hồi nhanh và độ chính xác cao, hướng đến ứng dụng thực tế trong các thiết bị tương tác thông minh

TÀI LIỆU THAM KHẢO

[1] TensorFlow (n.d.), "Video classification tutorial", https://www.tensorflow.org/tutorials/video/video_classification.

[2] saba99 (n.d.), "3D-CNN", https://medium.com/@saba99/3d-cnn-4ccfab119cc2.

[3] Medium (n.d.), "3D-CNN+LSTM: Deep Neural Networks for No-Reference Video Quality Assessment".

[4] aicandy.vn (n.d.), "Recurrent Neural Network (RNN) – Ứng dụng và cách hoạt động", https://aicandy.vn/recurrent-neural-network-rnn-ung-dung-va-cach-hoat-dong/.

[5] NT Tuấn (n.d.), "Bài 14: Long Short-Term Memory (LSTM)", https://nttuan8.com/bai-14-long-short-term-memory-lstm/.

[6] GeeksforGeeks (n.d.), "Deep Learning | Introduction to Long Short-Term Memory", https://www.geeksforgeeks.org/deep-learning-introduction-to-long-short-term-memory/.

[7] Dive into Deep Learning (D2L) Vietnam (n.d.), "GRU - Gated Recurrent Unit", https://d2l.aivivn.com/chapter_recurrent-modern/gru_vn.html.

[8] Viblo (n.d.), "Đánh giá model trong Machine Learning", https://viblo.asia/p/danh-gia-model-trong-machine-learing-RnB5pAq7KPG.

[9] ResearchGate (n.d.), "Basic 3D-CNN architecture: the 3D filter is convolved with the video in three dimensions", https://www.researchgate.net/figure/Basic-3D-CNN-architecture-the-3D-filter-is-convolved-with-the-video-in-three-dimensions_fig1_330912338.

[10] MDPI (n.d.), "A 3DCNN-LSTM Multi-Class Temporal Segmentation for Hand Gesture Recognition".

[11] TechScience (n.d.), "Dynamic Hand Gesture Recognition Using 3D-CNN and LSTM Networks".

[12] Materzyńska, J., Woźniak, M., & Świtoński, A. (2019), "The Jester Dataset: A Large-Scale Video Dataset of Human Gestures", Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops (ICCVW), 0, 3045–3054.